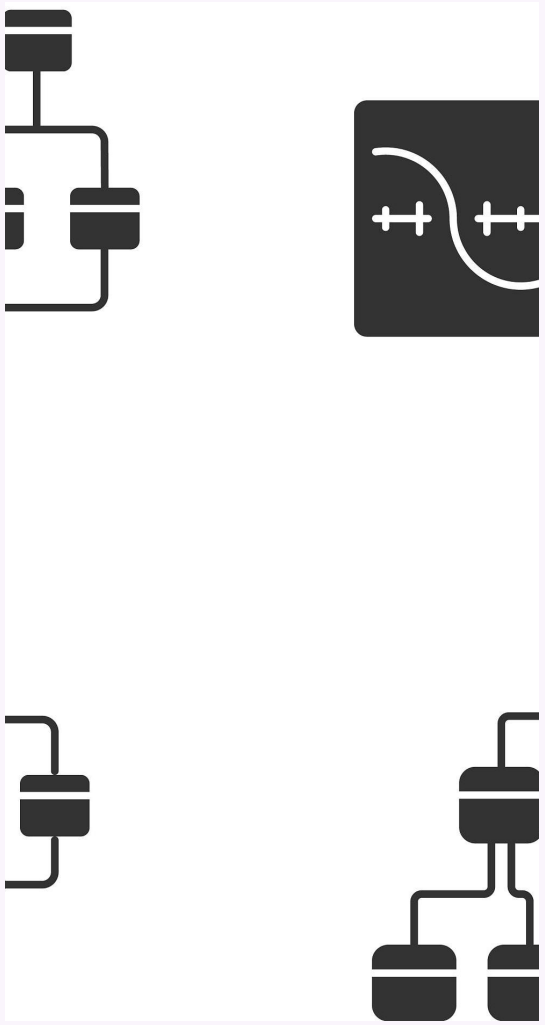
Recurrent Neural Networks and Natural Language Processing

01

Fundamentals of Recurrent Neural Networks

Limitations of Feedforward Neural Networks



Disordered node connections

In feedforward neural networks, the nodes between each layer are independent and lack connections, which limits the flow of information and makes it difficult to capture long-term dependencies between complex data.

Fixed Input/Output

The input/output size of feedforward neural networks is fixed, making it difficult to flexibly adapt to data sequences of different lengths, which limits their ability in handling variable data.

Missing word order model

When processing language data, feedforward neural networks have fixed positions for vocabulary nodes, making it difficult to model word order, which is a key element in language understanding.

Basic principles of recurrent neural networks

Recurrent neural network processing sequence

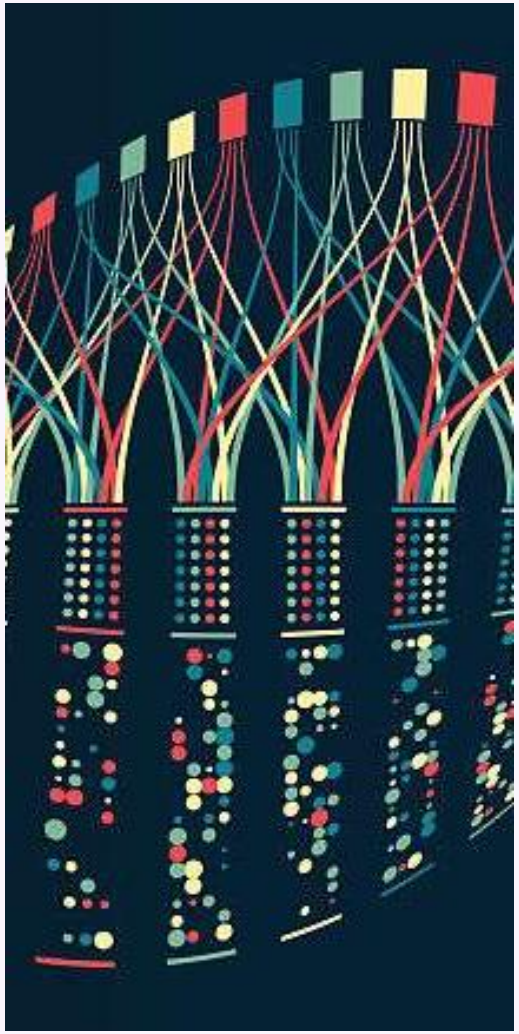
Recurrent neural networks introduce self feedback connections to enable neurons to process and retain historical input information, effectively handling sequence data processing tasks.

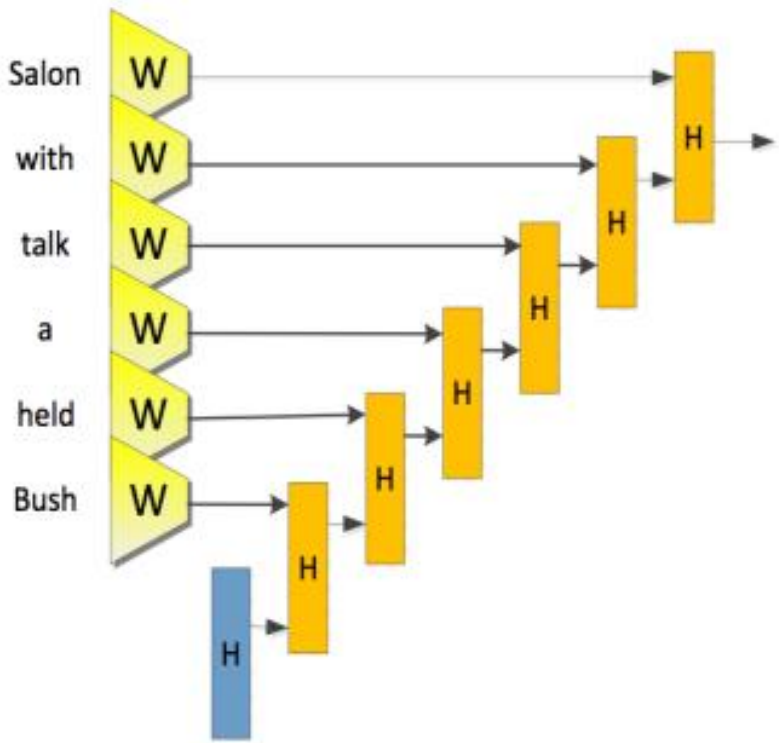
Inspired by the structure of biological networks

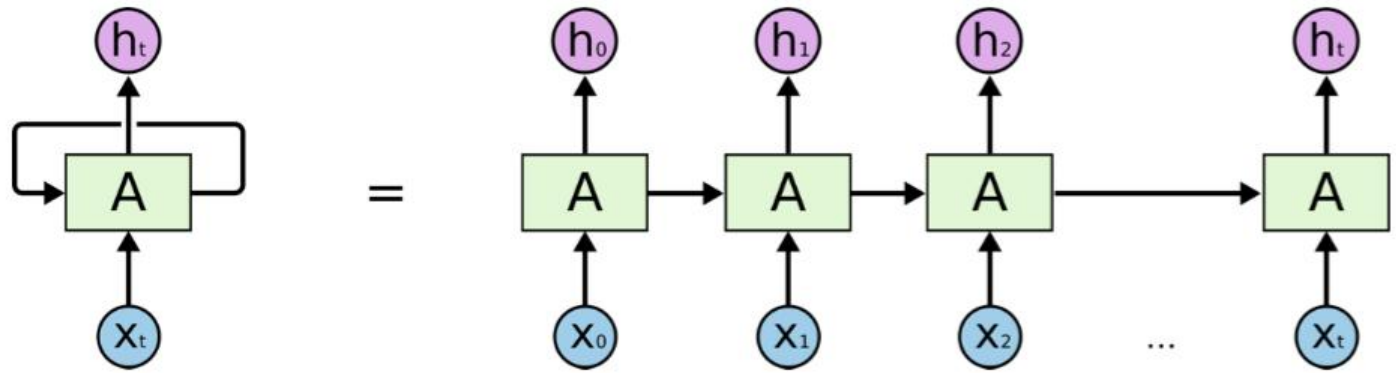
The design of RNNs is inspired by the structure of biological neural networks, which can more naturally process data with temporal dependencies, such as time series or text data.

Wide application areas

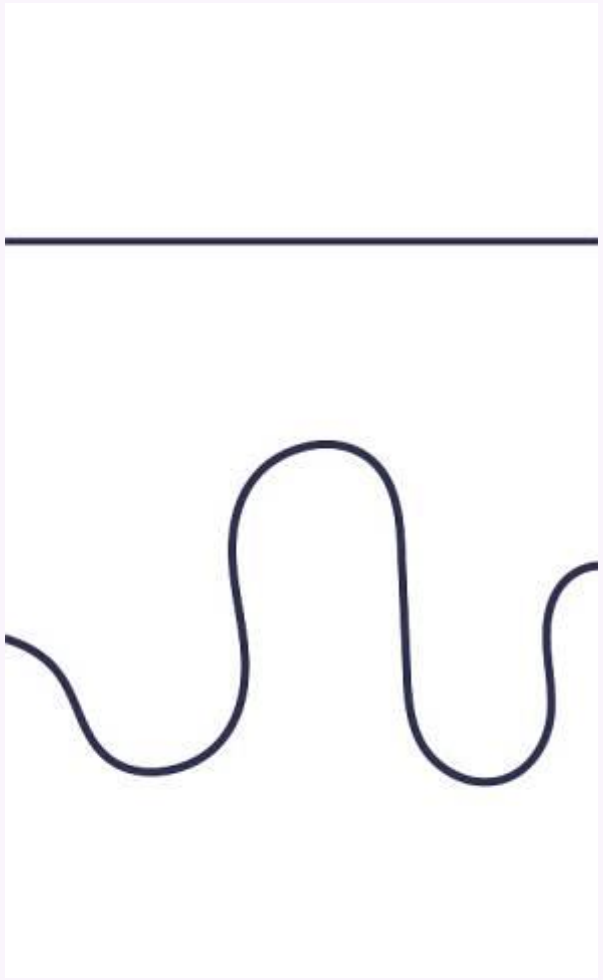
Recurrent neural networks have a wide range of applications in speech recognition, image processing, language modeling, and natural language generation, and their powerful sequence processing capabilities play a key role.




Salon
with
talk
a
held
Bush
W
H


h_t
A
x_t
=
h_0
h_1
h_2
h_t
A
x_0
x_1
x_2
...
x_t

Simple Recurrent Neural Network Structure



Current and past states

At time t, the input is xt, and the hidden state (activity of hidden neurons) is ht. ht is not only related to the input of the current time step, but also to the hidden state of the previous time step.

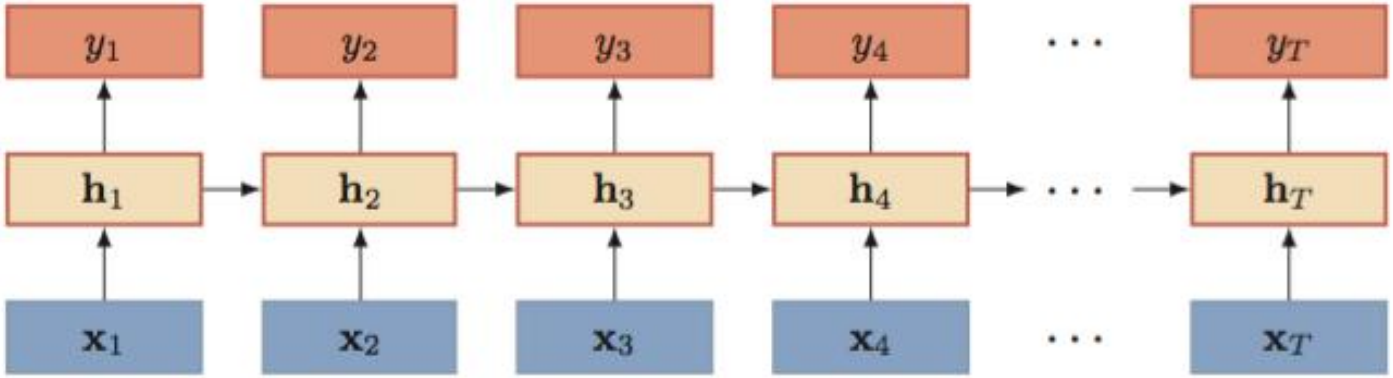
RNN hidden layer update

The hidden layer state is updated based on the current input and the previous hidden layer state, and is processed through nonlinear functions (such as S-shaped or hyperbolic tangent function tanh) to achieve nonlinear transformation of information.

Nonlinear activation is important

Nonlinear activation functions (such as tanh) are crucial in RNNs, as they not only introduce nonlinear factors to enhance the network's expressive power, but also help the network learn complex data representations.

$$\boldsymbol{h}_t = f(\boldsymbol{U}\boldsymbol{h}_{\boldsymbol{t-1}} + \boldsymbol{W}\boldsymbol{x}_{\boldsymbol{t}} + b) \qquad \boldsymbol{y}_t = softmax(\boldsymbol{W}^{(\boldsymbol{S})}\boldsymbol{h}_t)$$

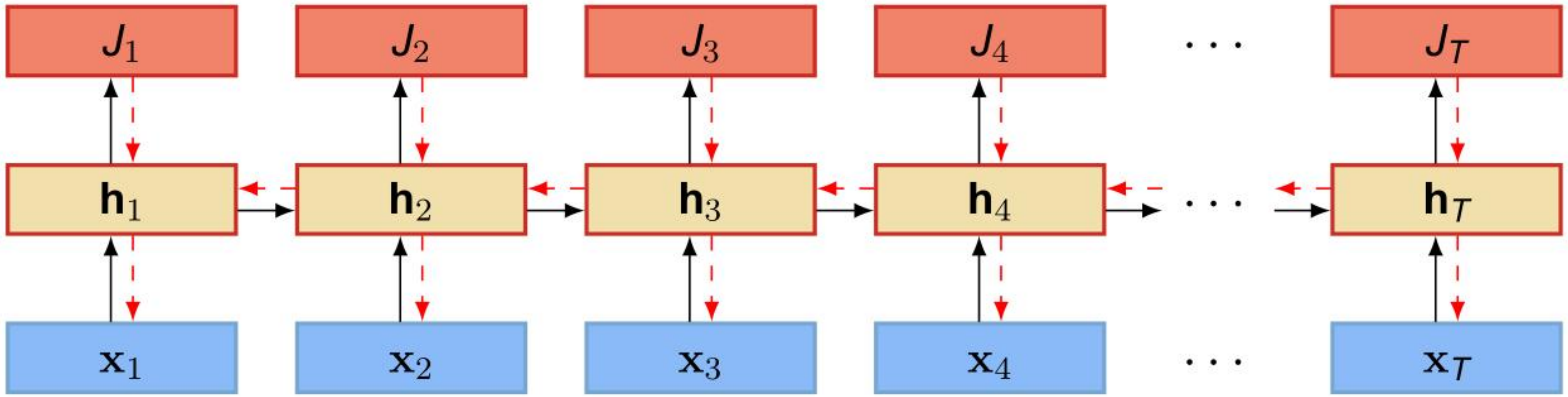


Gradient Calculation and BPTT Algorithm

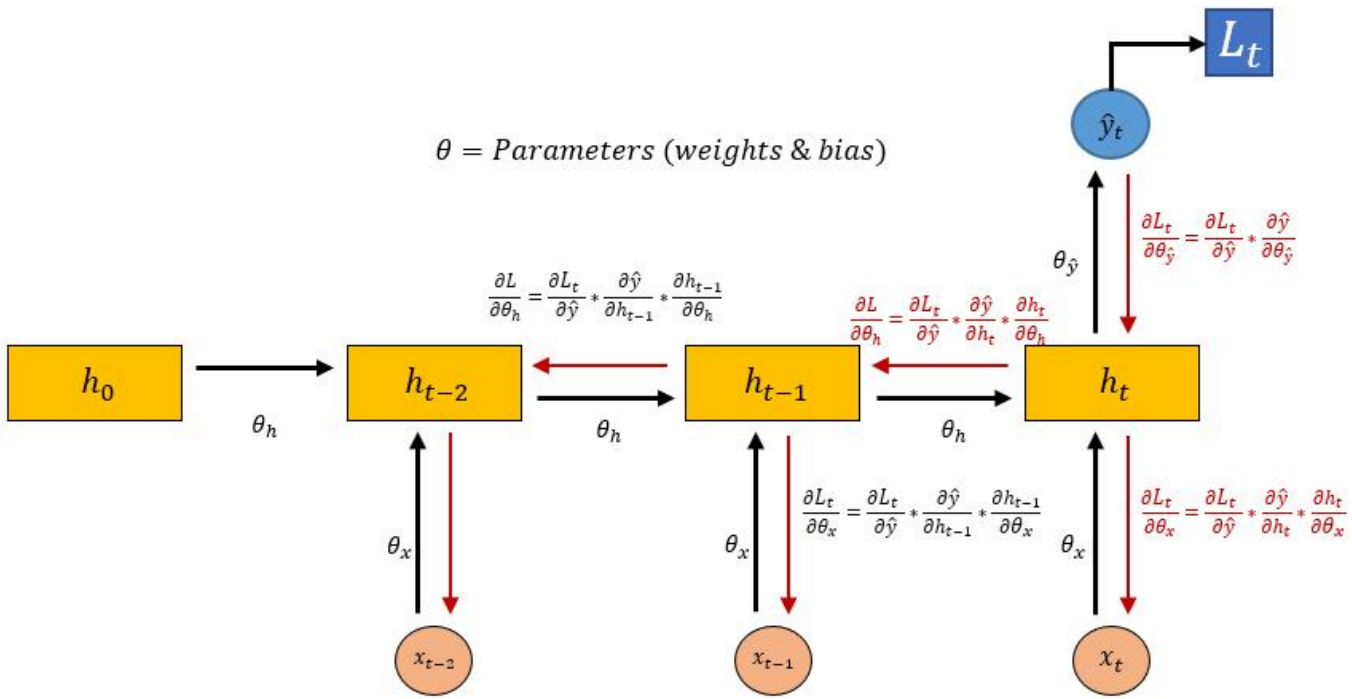
- RNN parameter training: Conducted through backpropagation over time (BPTT) algorithm. BPTT will unfold RNN over time to create a deep feedforward network representation, which is used to calculate gradients relative to RNN parameters.

- Gradient vanishing problem: The gradient vanishing problem is particularly prominent in RNNs, where as the time step increases, the magnitude of gradient updates sharply decreases, making it difficult for the network to learn long-term dependencies. This is due to the exponential weight reduction caused by multiplying multiple numbers less than 1.

- Addressing the issue of long-term dependencies: Suggested solutions include introducing different weight mechanisms to each time step, or using architectures with gating mechanisms such as LSTM/GRU, which can effectively alleviate the problem of gradient vanishing and enhance the network's ability to learn long-term dependencies.



P. J. Werbos. “Backpropagation through time: what it does and how to do it”. In: Proceedings of the IEEE 10 (1990).

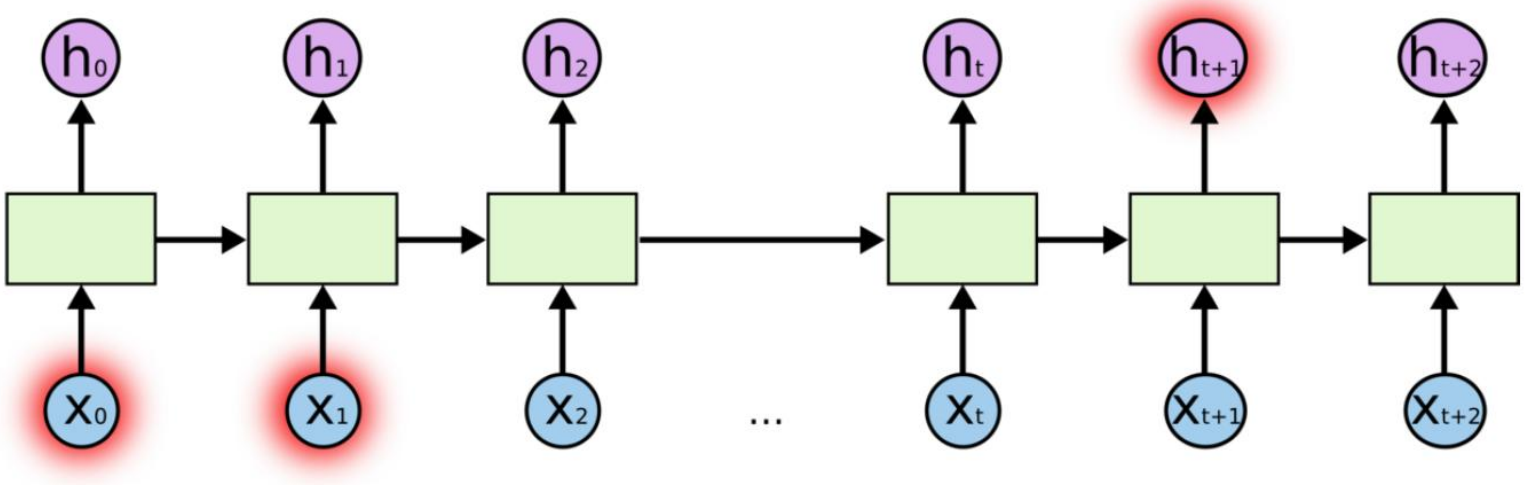


02

Long Short Term Memory Network

Long-term dependency in Simple RNN

- The cat, which already ate a bunch of food, (was) full.
- The cats, which already ate a bunch of food, (were) full.



Analysis of LSTM Network Structure

LSTM Unit Structure

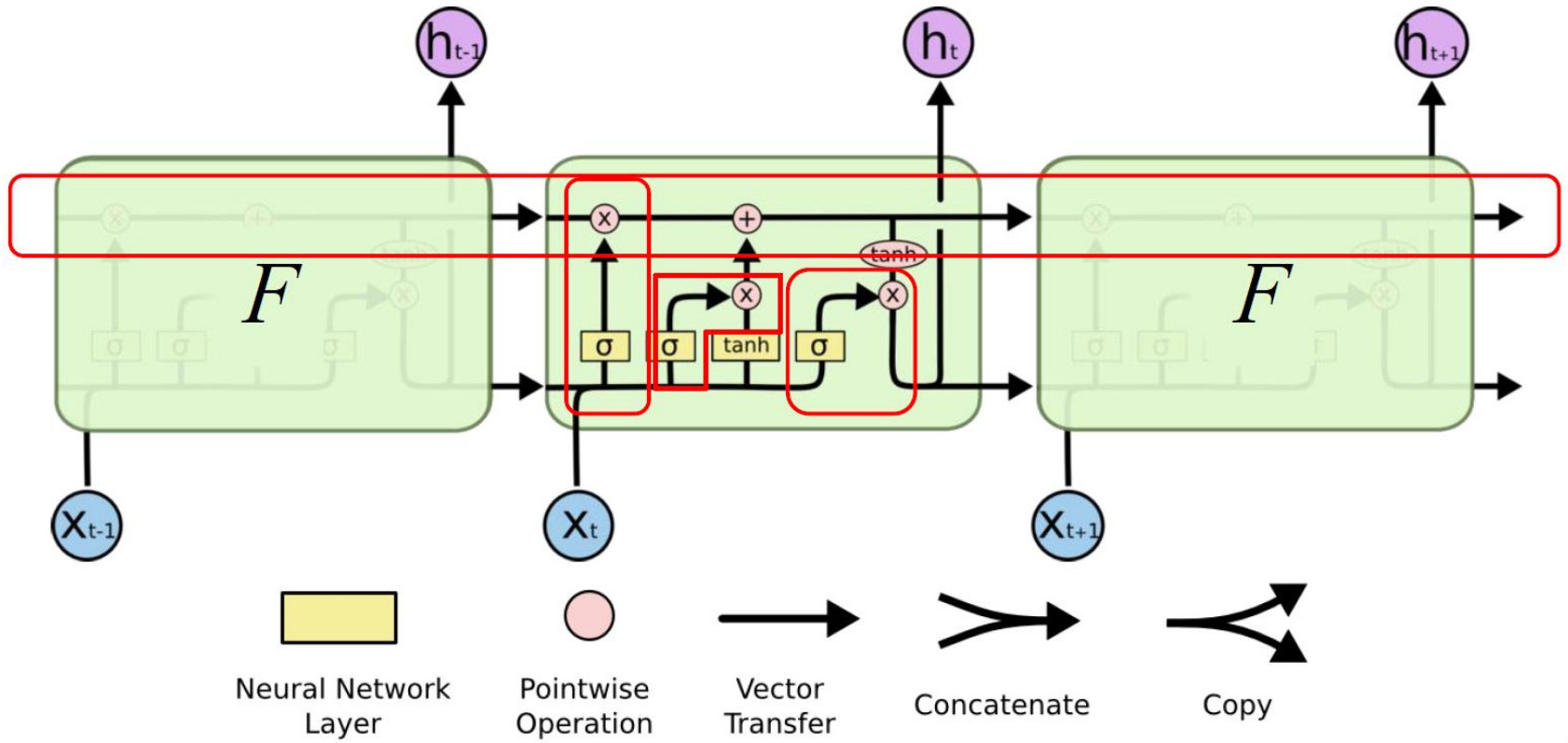
The LSTM network consists of a series of LSTM units, each unit containing four key components: forget gate, input gate, output gate, and cell state. These four parts work together.

Gated mechanism

LSTM controls the storage and forgetting of information through a gating mechanism. The forget gate regulates the discarding of historical information, the input gate controls the absorption of new information, and the output gate determines the amount of information output by the cell state.

Memory ability

LSTM cell states act as a memory warehouse, storing key information from sequence history. By regulating the input gate and forget gate, cell states can preserve key details of long-term dependencies.


h_{t-1}
h_t
h_{t+1}
F
F
σ
σ
tanh
σ
tanh
X_{t-1}
X_t
X_{t+1}
Neural Network Layer
Pointwise Operation
Vector Transfer
Concatenate
Copy

Forgetting Gate and Input Gate Mechanism



Forgetting Gate Mechanism

The forgetting gate determines whether to forget information in the cell state based on the current input and the hidden state of the previous step, achieving selective discarding of historical information and avoiding information overload.



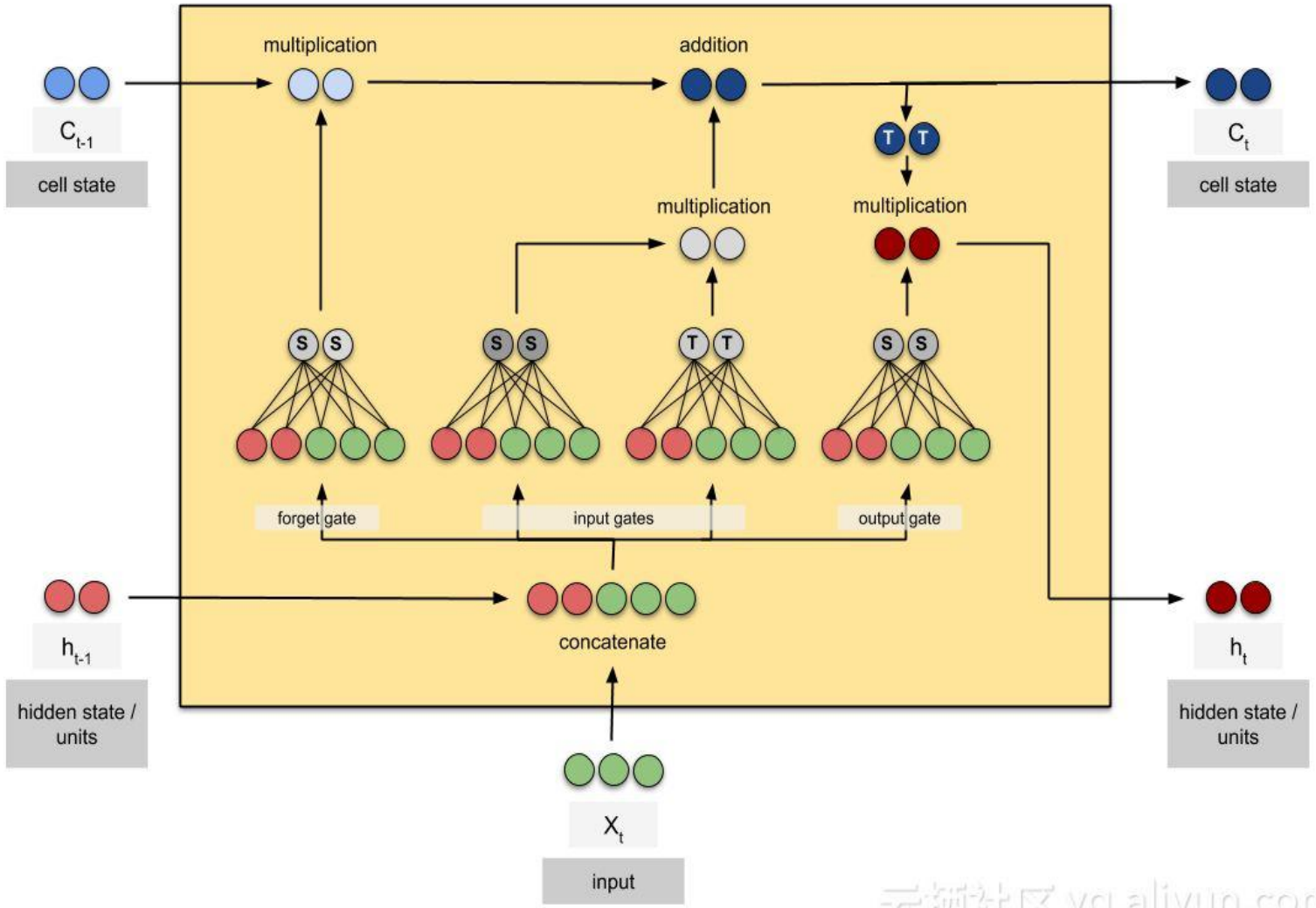
Input gate mechanism

The input gate is responsible for controlling the degree to which new information flows into the cell state, regulating it through an activation function to ensure that important information is effectively stored while preventing interference from irrelevant information.



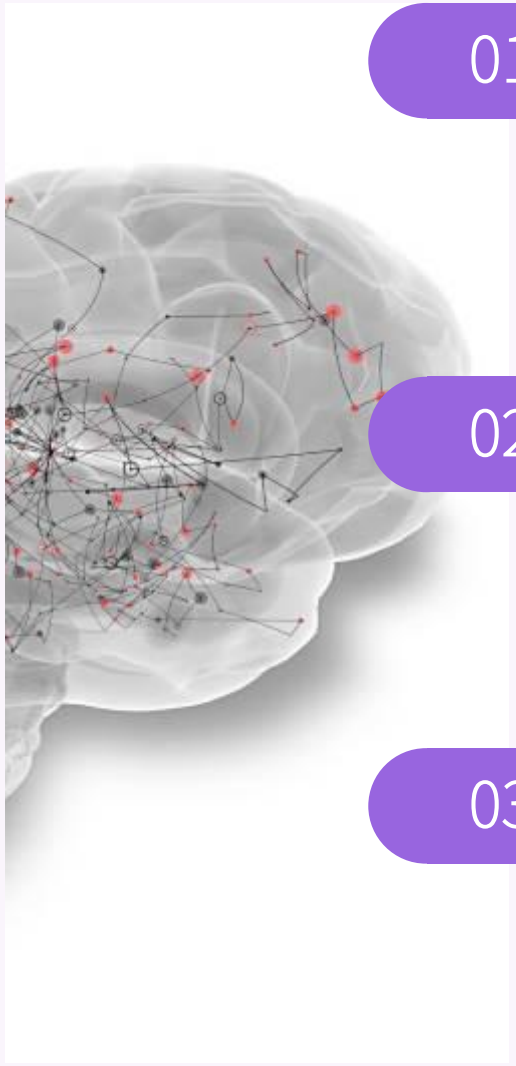
Writing mechanism

Through the synergistic effect of input gates and forget gates, LSTM is able to write new memories in the cellular state, and this process is incremental, ensuring that the network can continuously learn new patterns.



云栖社区 yq.aliyun.com

Working principle of LSTM unit



01

Forgetting and input coordination

Within the LSTM unit, the forget gate and input gate work together to balance the forgetting of historical information and the acceptance of new information, ensuring that critical information is not lost while promoting continuous learning.

02

Output gate regulation

The output gate determines the output of the current time step based on the cell state and the hidden state of the previous time step, effectively regulating the information flow and ensuring that the LSTM unit can transmit important information in a timely manner.

03

Cell state purification

Through strict regulation of the forget gate and input gate, the cell state is purified, unnecessary information is removed, key memory is retained, and the processing ability of LSTM network for sequence data is enhanced.

03

Gate controlled cycle unit

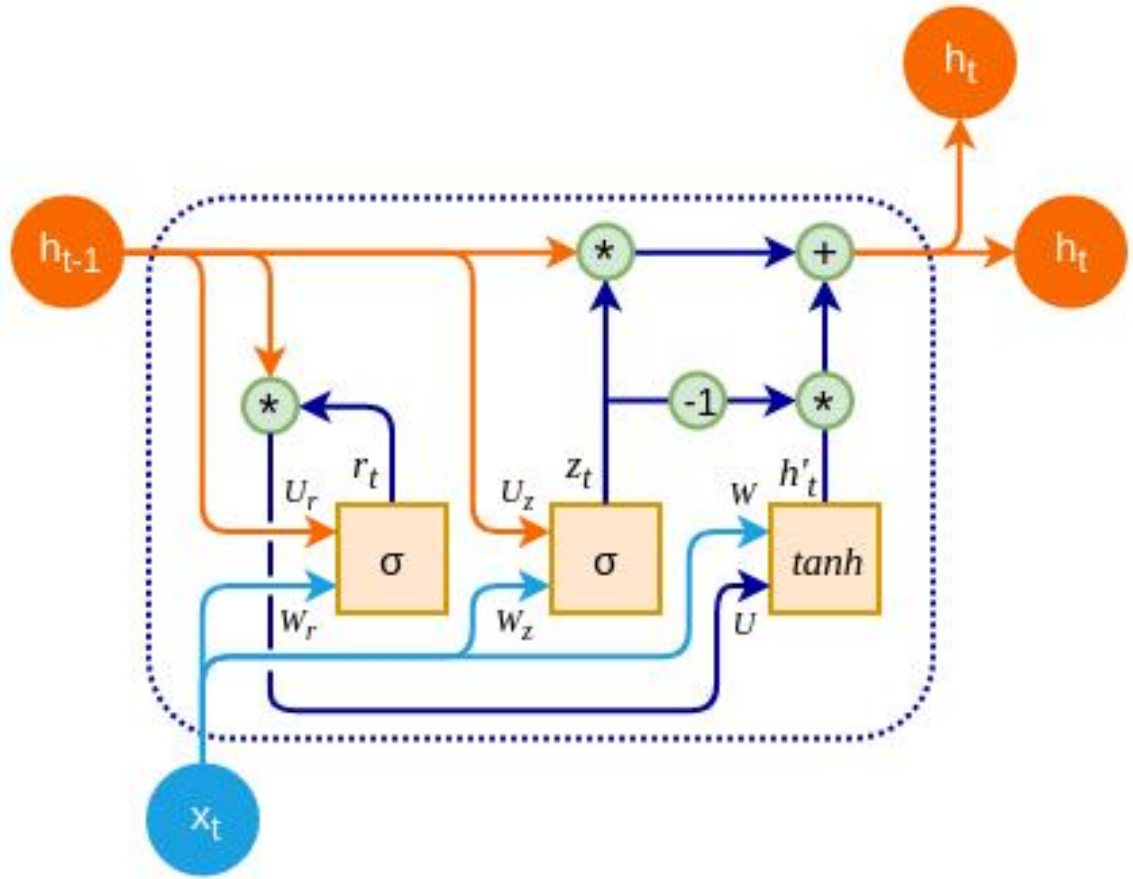
Characteristics of GRU architecture composition

- The essence of GRU architecture: GRU achieves efficient information filtering and transmission by introducing reset and update gates. Its architecture is streamlined, containing only two core components. Compared to LSTM, it reduces the number of parameters and improves training efficiency.

- Reset gate and update gate: Reset gate controls the degree of information forgetting and helps the model ignore unimportant information; The update gate determines the proportion of new information retained, promoting continuous learning of the model. The two work together to optimize sequence data processing.

- Hidden state update: In GRU, the hidden state is updated based on the current input and the previous hidden state, ensuring effective transmission and integration of information, allowing the model to dynamically adjust its internal state to cope with changes.

- Simplifying Long Term Dependency Handling: By simplifying the complex structure of LSTM, GRU reduces unnecessary components such as forget gates and candidate hidden state computation gates, thereby simplifying the handling of long-term dependencies and improving the training efficiency of the model.


h_t
h_{t-1}
*
+
h_t
*
-1
*
r_t
z_t
h'_t
U_r
U_z
W
σ
σ
tanh
W_r
W_z
U
x_t

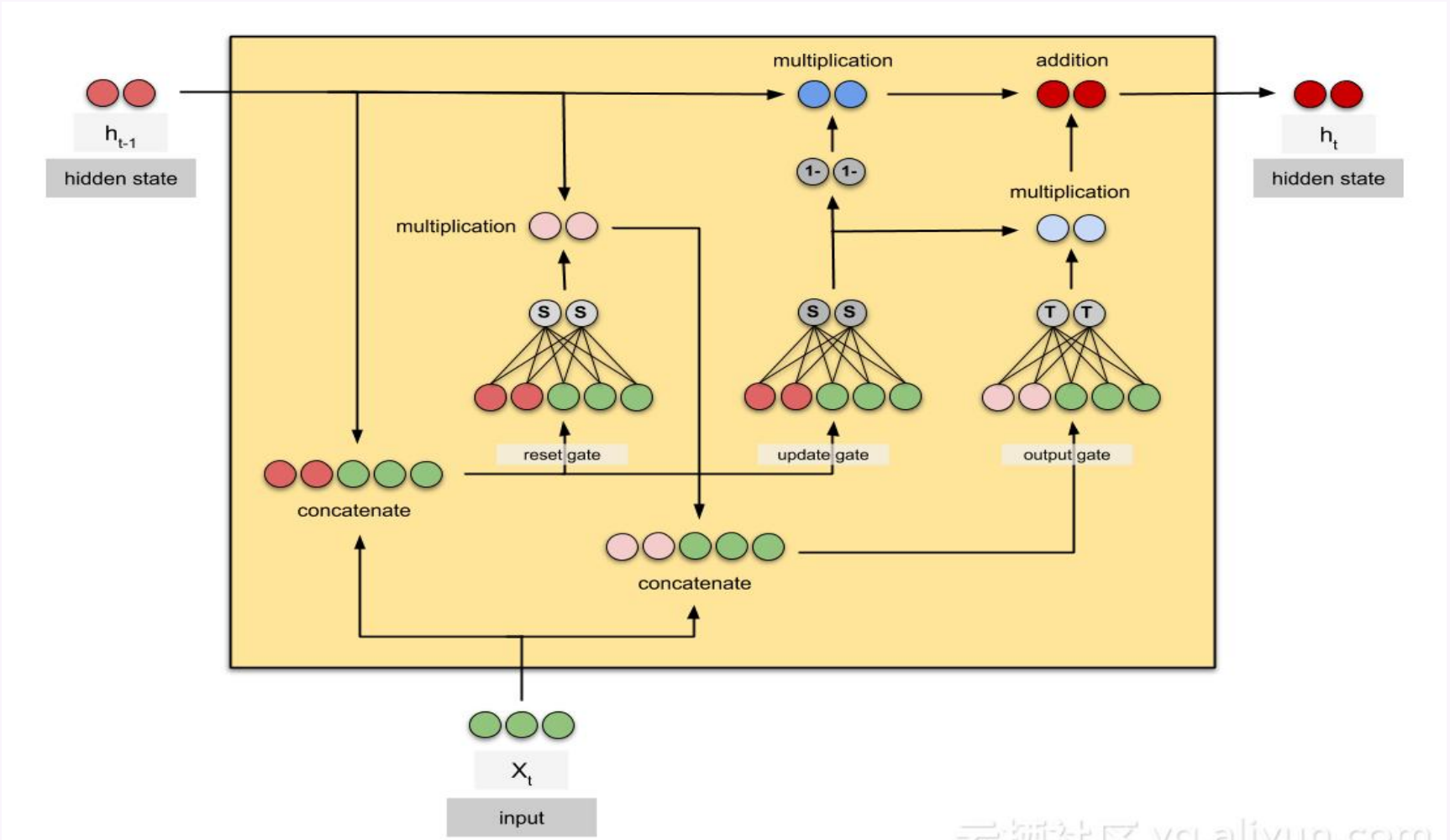
Detailed explanation of GRU unit structure

- GRU unit structure: The hidden layer neurons control the selection of new information and the forgetting of old information through two mechanisms, update gates and reset gates, respectively, thereby achieving effective information updating and forgetting and enhancing the network's adaptability to dynamic environments.

- The function of the update gate: is to dynamically determine the proportion of new information retention based on the current input and the previous hidden state, ensuring that important information is not forgotten, while promoting the model to continuously learn new content and improve its ability to process sequence data.

- Reset door function: Reset the degree of forgetting of door control information, help the model ignore unimportant information, and reduce the burden on the model. By resetting the on/off state of the door, the model can focus more on the current input information and improve learning efficiency.

- Hidden state update: The hidden state is updated based on the current input, previous hidden state, update gate, and reset gate output. This update method ensures effective transmission and integration of information, enabling the model to dynamically adjust its internal state.



云栖社区 yq.aliyun.com

Comparative analysis of GRU and LSTM

- Comparison between GRU and LSTM: GRU is known for its simpler architecture and more efficient training speed, especially suitable for large sequence data processing tasks, while LSTM is favored for its powerful long-term dependency processing capabilities.
- The training advantages of GRU: Due to its relatively simple architecture and fewer parameters, GRU typically has faster training speed and higher training efficiency when processing sequence data, making it suitable for scenarios that require fast response.
- Applicability of LSTM: When faced with tasks that require precise capture of long-term dependencies, such as certain natural language processing or time series prediction problems, LSTM's table performance is usually superior to GRU, and its powerful ability to handle long-term dependencies is key.
- Choice suggestion: Whether to choose GRU or LSTM depends on the specific task requirements. For scenarios that require fast processing of large sequence data and low long-term dependency requirements, GRU is a better choice; On the contrary, LSTM is more suitable.

05

Advanced RNN technology

Advantages of Stacked RNN Structure

1

Complex information processing

Through a multi-layer hidden layer structure, Stacked RNN can efficiently capture and process complex time series dependencies, significantly improving the model's ability to model long-term dependencies.

2

Feature level abstraction

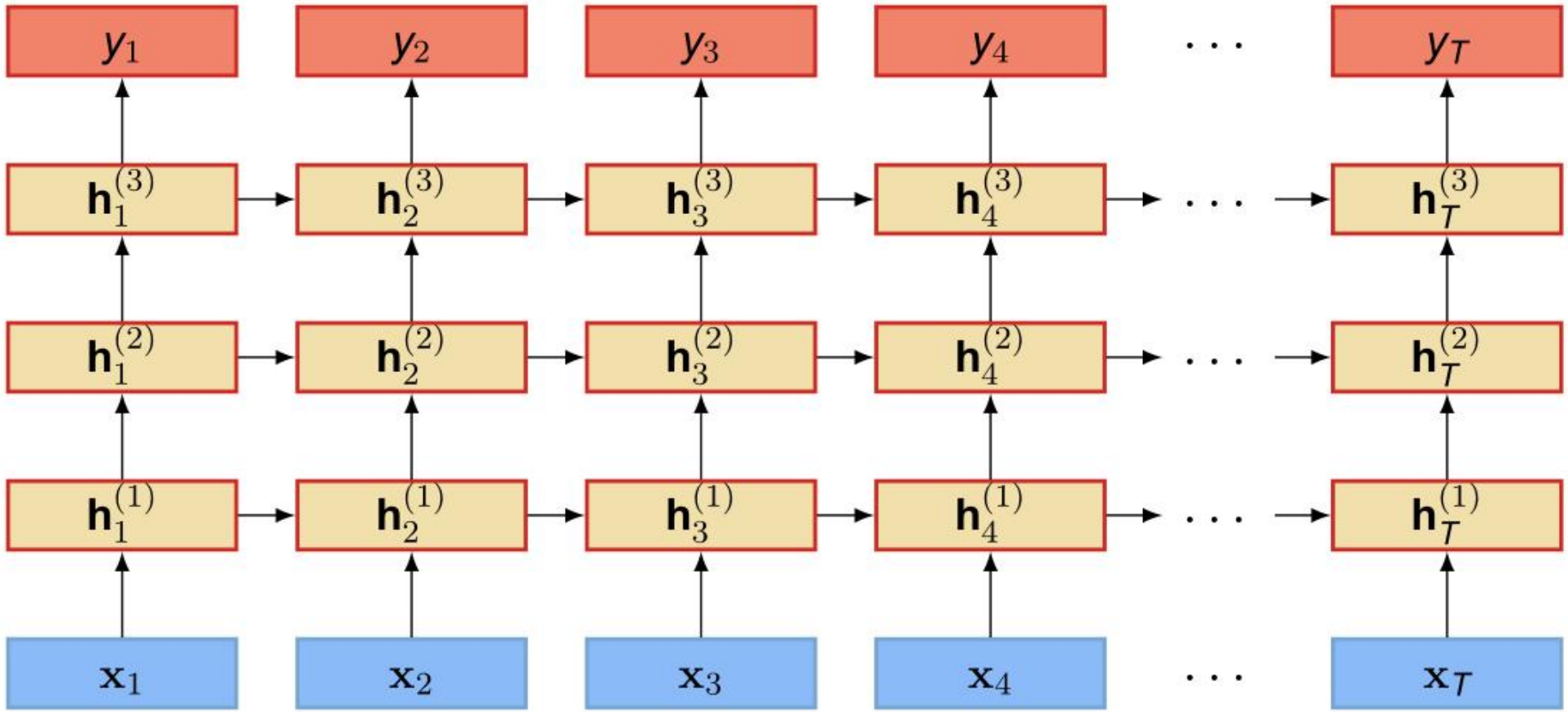
Each hidden layer can learn and extract different levels of features from input data, thereby achieving feature abstraction and hierarchical representation, enhancing the expression and generalization ability of the model.

3

Memory management optimization

Stacked RNN achieves more efficient memory management and utilization through the design of multiple hidden layers, optimizing the storage efficiency of the model while improving processing speed.




y_1
y_2
y_3
y_4
y_T
$\mathbf{h}_1^{(3)}$
$\mathbf{h}_2^{(3)}$
$\mathbf{h}_3^{(3)}$
$\mathbf{h}_4^{(3)}$
$\mathbf{h}_T^{(3)}$
$\mathbf{h}_1^{(2)}$
$\mathbf{h}_2^{(2)}$
$\mathbf{h}_3^{(2)}$
$\mathbf{h}_4^{(2)}$
$\mathbf{h}_T^{(2)}$
$\mathbf{h}_1^{(1)}$
$\mathbf{h}_2^{(1)}$
$\mathbf{h}_3^{(1)}$
$\mathbf{h}_4^{(1)}$
$\mathbf{h}_T^{(1)}$
$\mathbf{x}_1$
$\mathbf{x}_2$
$\mathbf{x}_3$
$\mathbf{x}_4$
$\mathbf{x}_T$

Working mechanism of bidirectional RNN

01

Bidirectional Information Capture

Bidirectional RNN is ingeniously designed to handle bidirectional inputs of sequences, ensuring that the model can comprehensively capture contextual information from the past and future, enhancing its understanding and predictive ability of sequence data.

02

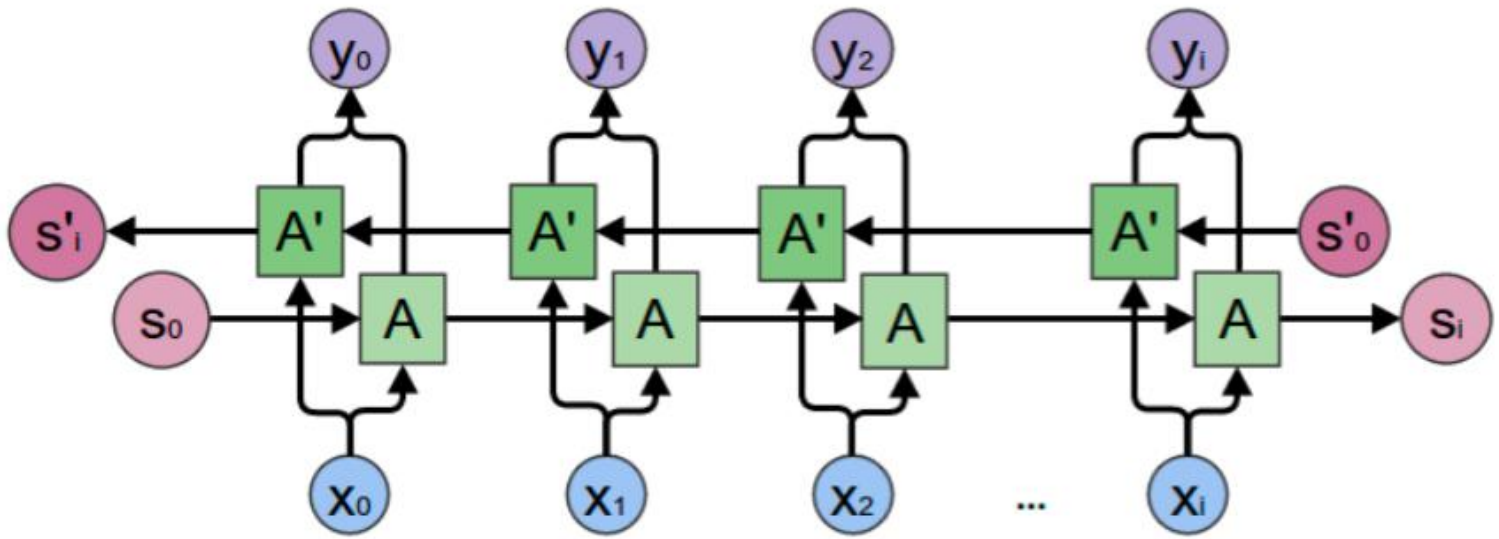
Hidden layer state concatenation

In Bidirectional RNN, the forward and backward hidden layer states are accurately concatenated to form a richer and more comprehensive sequence representation, providing strong support for subsequent decision-making or prediction.

03

Context information utilization

By integrating bidirectional hidden states, Bidirectional RNN can fully utilize past and future context information, significantly improving the accuracy and intelligence of the model in processing sequence data.


y0
y1
y2
yi
s'i
A'
A'
A'
A'
s'0
s0
A
A
A
A
si
x0
x1
x2
...
xi

Principles of Seq2Seq Model

Encoder decoder architecture

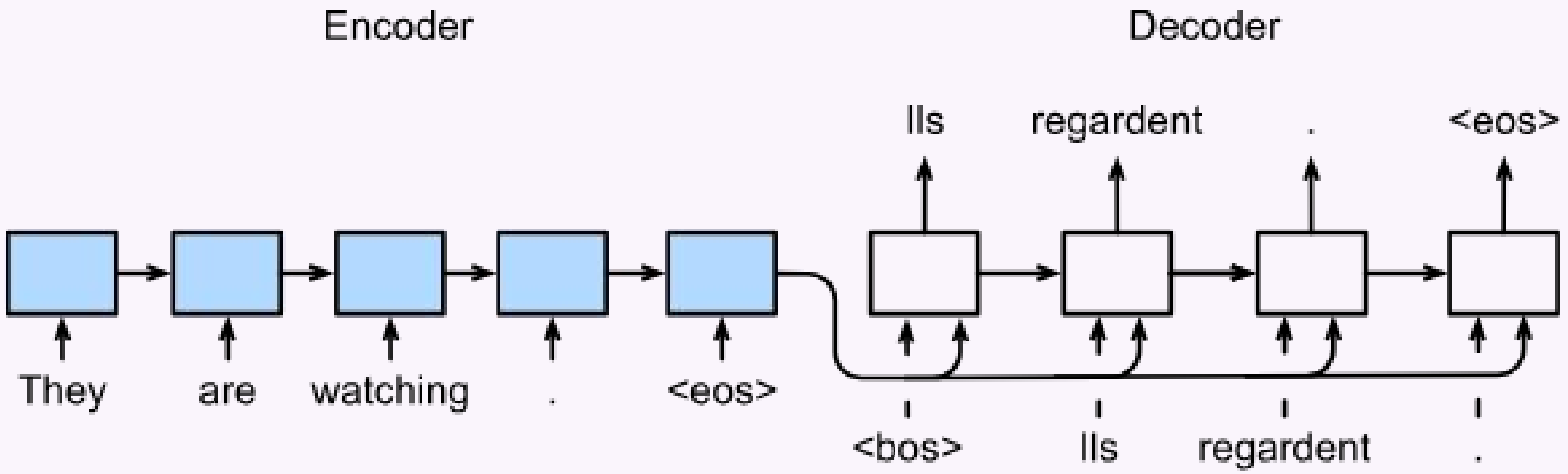
The Seq2Seq model adopts an encoder decoder architecture, where the encoder is responsible for converting the input sequence into a fixed dimensional vector representation, and the decoder generates an output sequence based on this vector.

Context Vector Extraction

In the Seq2Seq model, the encoder processes the input sequence, extracts and generates a context vector containing key information, which is then used as the initial input of the decoder to guide it in generating the output sequence.

Variable length processing

The Seq2Seq model has excellent flexibility and can easily handle input and output sequences of any length. Through the collaborative work of the encoder and decoder, it achieves precise sequence to sequence conversion.


Encoder
Decoder
Ils
regardent
.
<eos>
They
are
watching
.
<eos>
<bos>
Ils
regardent
.

05

The Application of RNN in NLP

Implementation of Text Classification Task

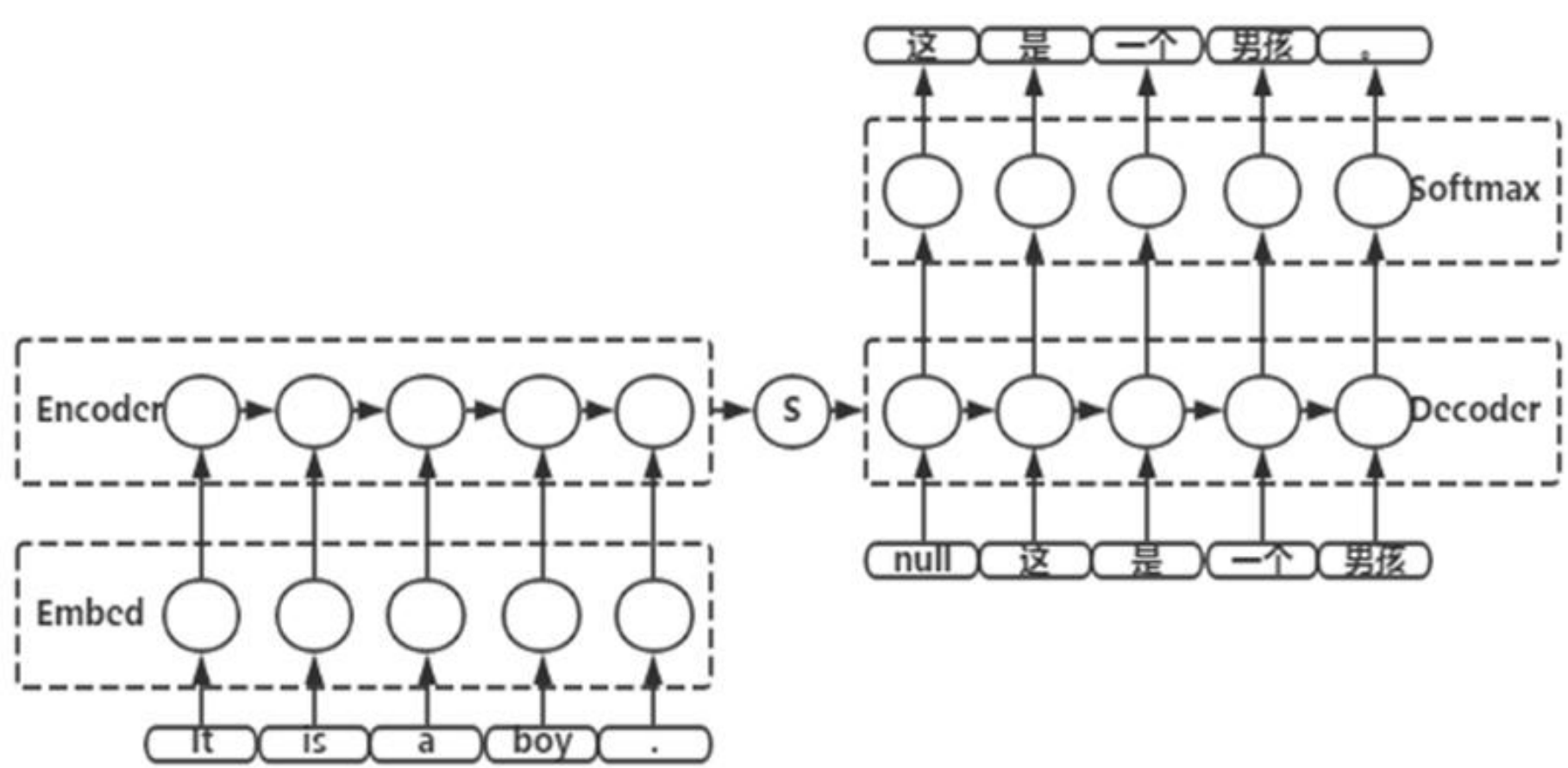
- Model selection: For text classification tasks, an RNN model suitable for processing sequential data, such as LSTM or GRU, can be selected to construct a classifier. By capturing sequence features, improve classification accuracy.
- Embedding layer design: In the model, word embeddings (such as Word2Vec) are used in the input layer to convert text into high-density numerical vectors, effectively preserving text information while solving the problem of high-dimensional sparsity in traditional one hot encoding.
- RNN layer construction: Stacking multiple layers of RNNs to construct deep networks, enhancing the model's ability to learn complex sequence dependencies. Each layer receives the output of the previous layer, learns different abstract level features, and enhances the model's expressive power.
- Output layer design: In the model output layer, for binary classification tasks, the sigmoid activation function is used to convert the model output into probability values; For multi class tasks, the softmax function is used for class prediction.

Automatic Summary Generation Technology

- Sequence to sequence architecture: Using the Seq2Seq model, automatic digest generation is achieved through an encoder decoder architecture. The encoder reads the original text, and the decoder generates a digest based on the encoder output to improve the quality of the digest.
- Attention mechanism: Introducing attention mechanism enables the decoder to dynamically focus on different parts of the original text when generating summaries, generating more accurate and comprehensive summaries, and improving the completeness and accuracy of the summaries.
- Copy mechanism: Introducing a copy mechanism that allows the model to directly copy key information from the original text when generating a summary, reducing information loss and redundancy, and improving the refinement and integrity of the summary.
- Coverage issue: Introduce coverage vectors or sets to record the generated summary information, avoid duplication and omission, ensure that the generated summary fully covers the key content of the original text, and improve the completeness and quality of the summary.

Machine Translation System Architecture

- Encoder decoder: Adopting an encoder decoder architecture, the encoder converts the input sequence into a fixed dimensional vector, and the decoder generates an output sequence based on the vector, achieving sequence to sequence conversion.
- Attention mechanism: Adding attention mechanism in the decoder enables the model to dynamically focus on different parts of the input sequence, generate more accurate translation results, and solve the encoder representation problem.
- Multi layer structure: Use multi-layer RNNs to build encoders and decoders to capture more complex sequence dependencies. Deep structure can enhance the learning ability of models and effectively respond to complex language phenomena.
- Bidirectional encoding: Introducing bidirectional encoders to process input sequences in both left to right and right to left directions, capturing bidirectional contextual information, enhancing the encoder's representational power, and ultimately improving translation quality.


这
是
一个
男孩
。
Softmax
Encoder
S
Decoder
Embed
null
这
是
一个
男孩
It
is
a
boy
.

06

Attention mechanism

Basic Principles of Attention

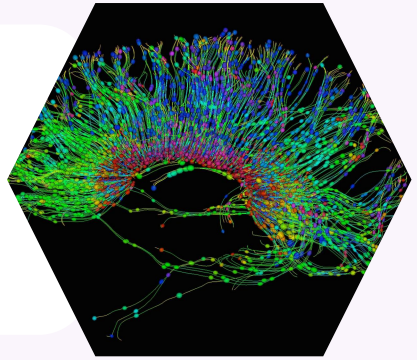


Attention mechanism core

The attention mechanism dynamically adjusts weights to enable the model to flexibly focus on important information when processing input sequences, improving the performance of sequence to sequence tasks.

Alignment and decoding

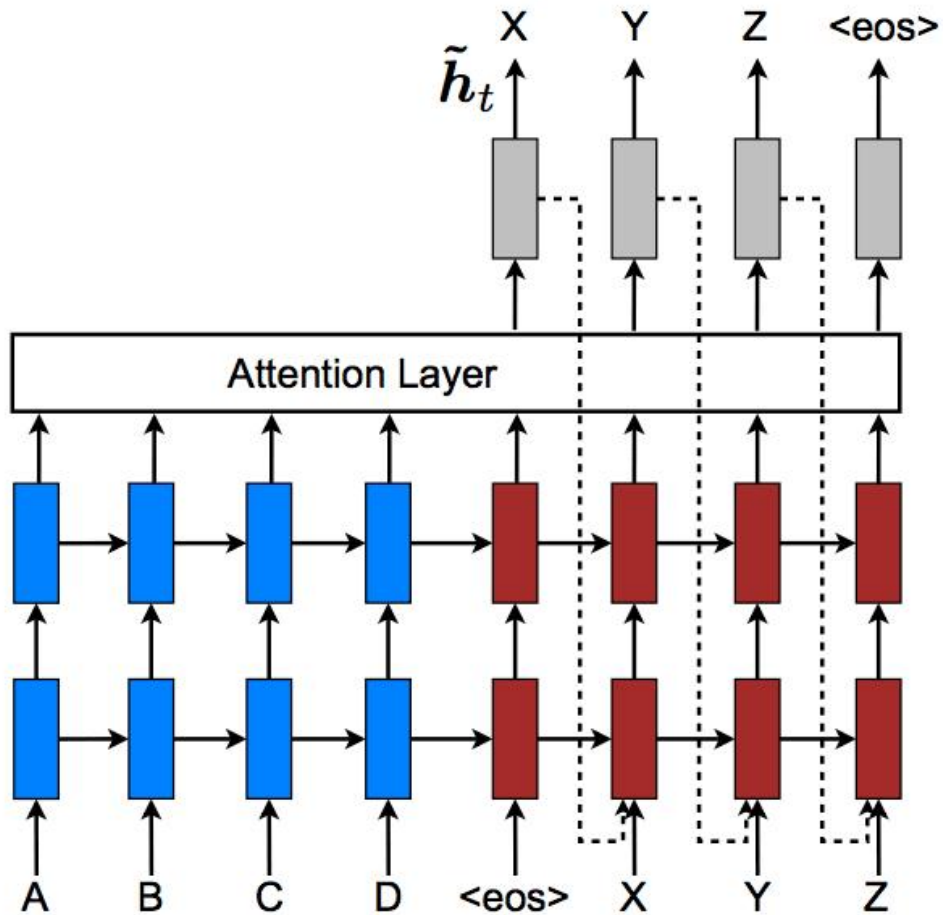
In the Seq2Seq model, the attention mechanism calculates the alignment score to ensure that the decoder can refer to the input part corresponding to the encoder when generating the output.





Challenges and Opportunities

Although implementing and fine-tuning attention mechanisms is challenging, gaining a deeper understanding of the problem and model can overcome the bottleneck issues of the Seq2Seq model and improve its performance.


X
Y
Z
<eos>
$\tilde{h}_t$
Attention Layer
A
B
C
D
<eos>
X
Y
Z

Attention weight calculation

Attention weight calculation

In the RNN attention mechanism, Ci is calculated based on all hidden states of the encoder and the hidden states of the decoder from the previous step.

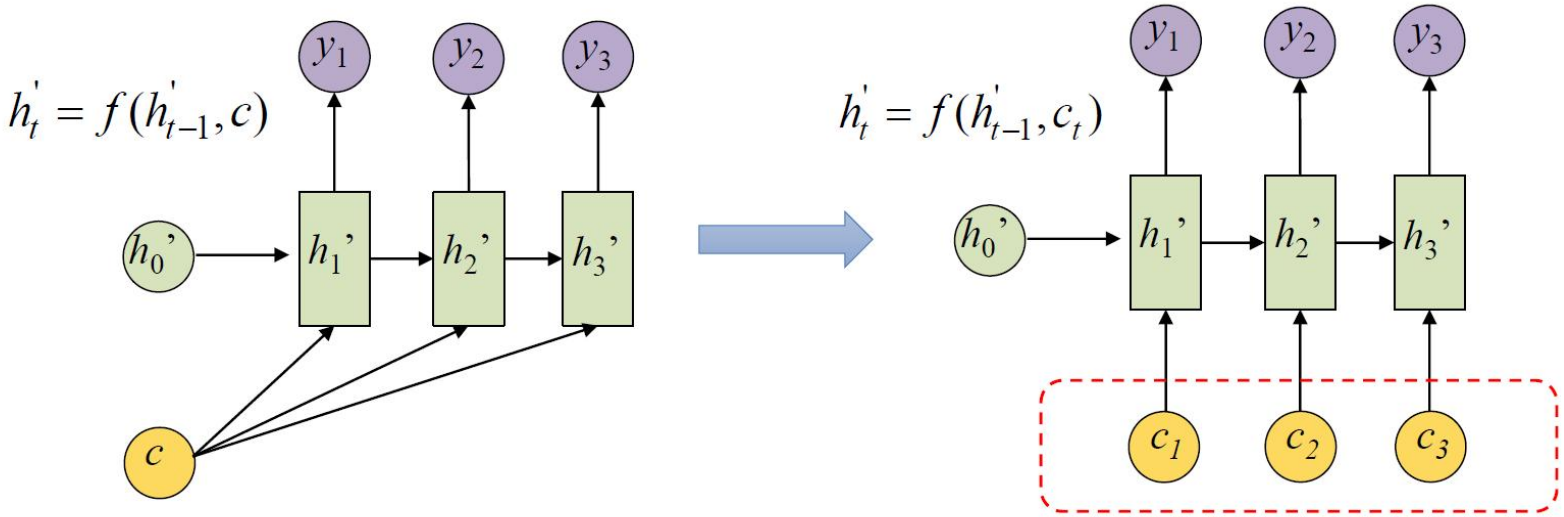
The impact of correlation

The calculation of Ci takes into account the correlation between each position in the input sequence and the current output time, and is obtained by calculating the attention score aij.

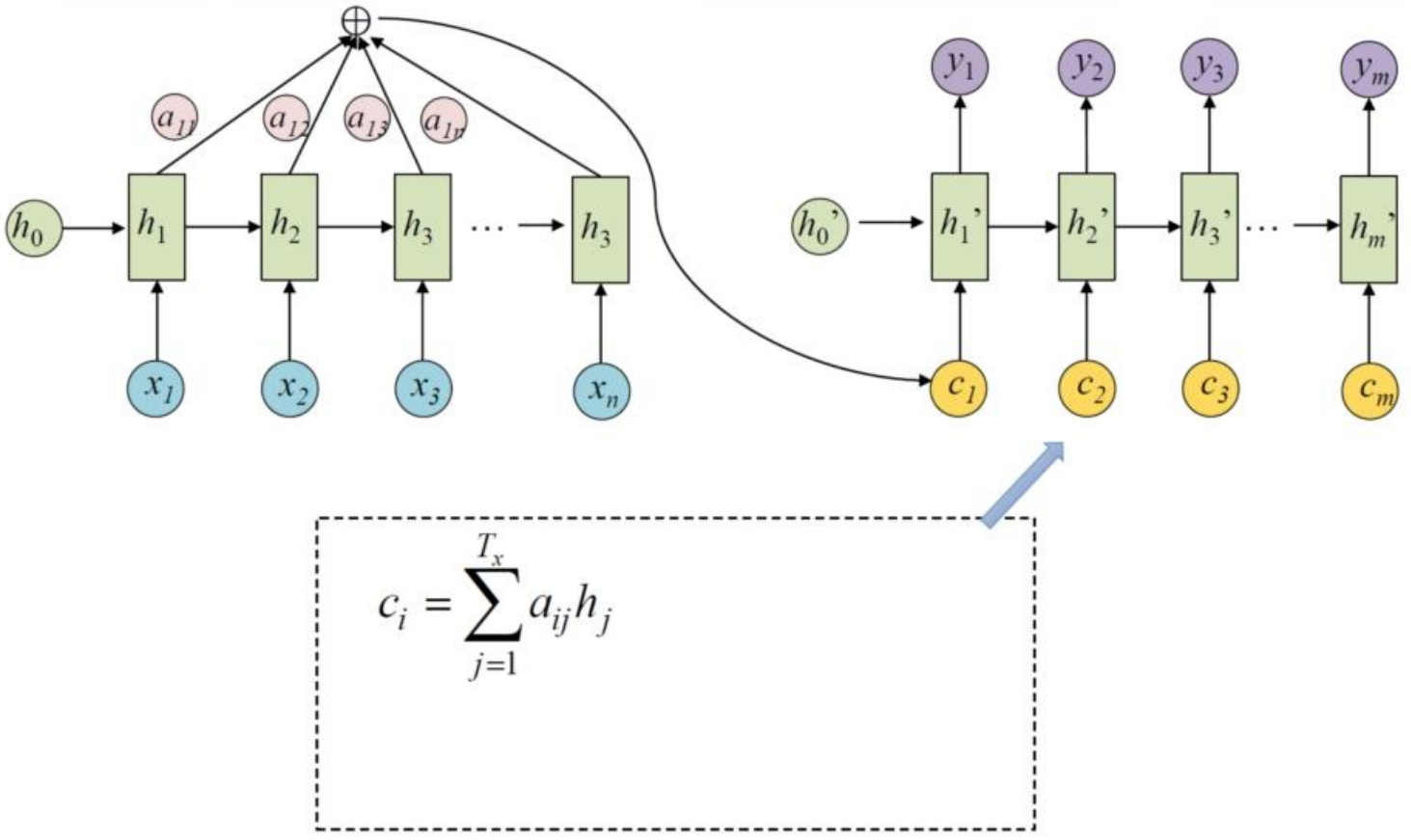
Adjustment of correlation weights

The calculation of aij is obtained by dot multiplying the output state of the decoder with each input state of the encoder, and normalizing it using a softmax function.

Attention RNN Example



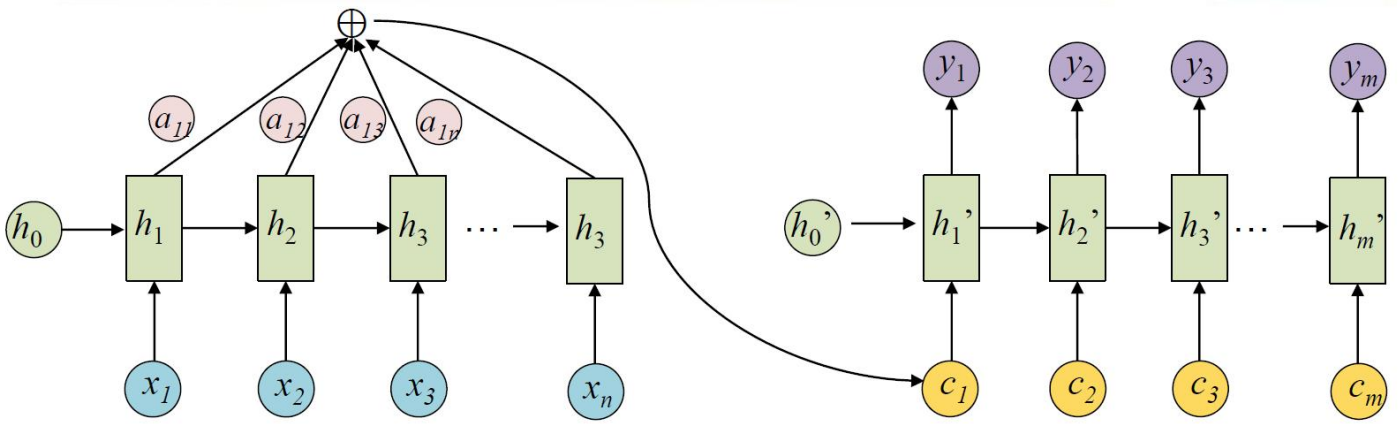
RNN Attention Example

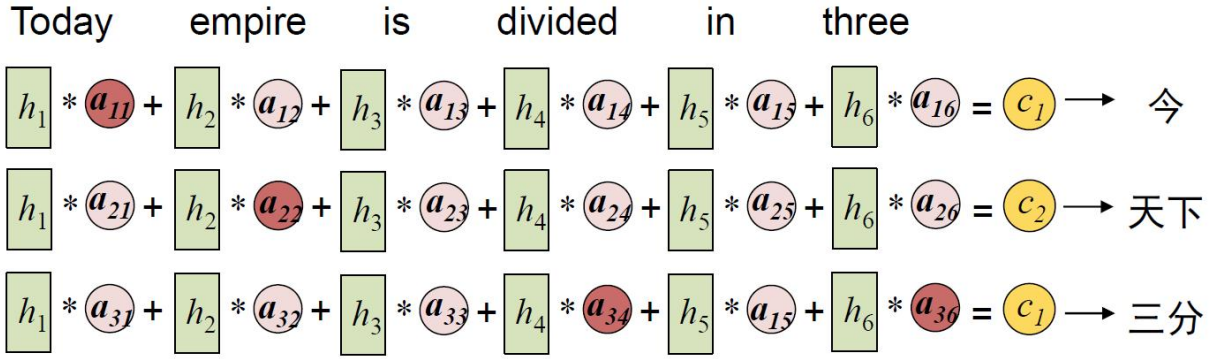


c is the weighted sum of the hidden states in the encoder, and a is the alignment probability between the target word y and the source word x.

RNN Attention Example?

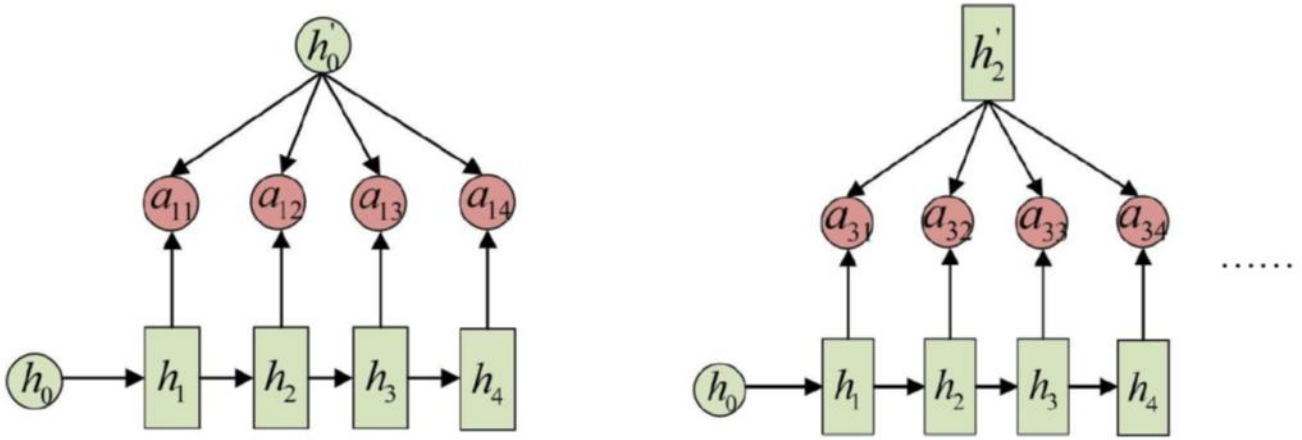
How to compute C?





RNN Attention Example

How to compute *a* ?



$$a_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})}, \quad e_{ij} = \varphi(h'_{i-1}, h_j) = V^T \tanh(Wh'_{i-1} + Uh_j)$$

- a_{ij} is the probability that the target word y will align with the source word x
- e_{ij} is the corresponding energy function
- φ is an alignment model that measures how well the i-position input matches the position-output

Improvement of Attention Mechanism



Introducing multi head attention

The multi head attention mechanism further enhances the model's ability to process sequential data by processing multiple attention functions in parallel, thereby improving the model's expressiveness and attention allocation.



Combining Transformer

Google's GNMT architecture combines Transformer's attention mechanism to better handle long-term dependencies through multi head self attention mechanism.



Simplifying GRU/LSTM structure

min GRU and min LSTM simplify the structure of traditional GRU and LSTM, reducing the number of parameters and improving training efficiency while maintaining good performance.

07

Practical application cases

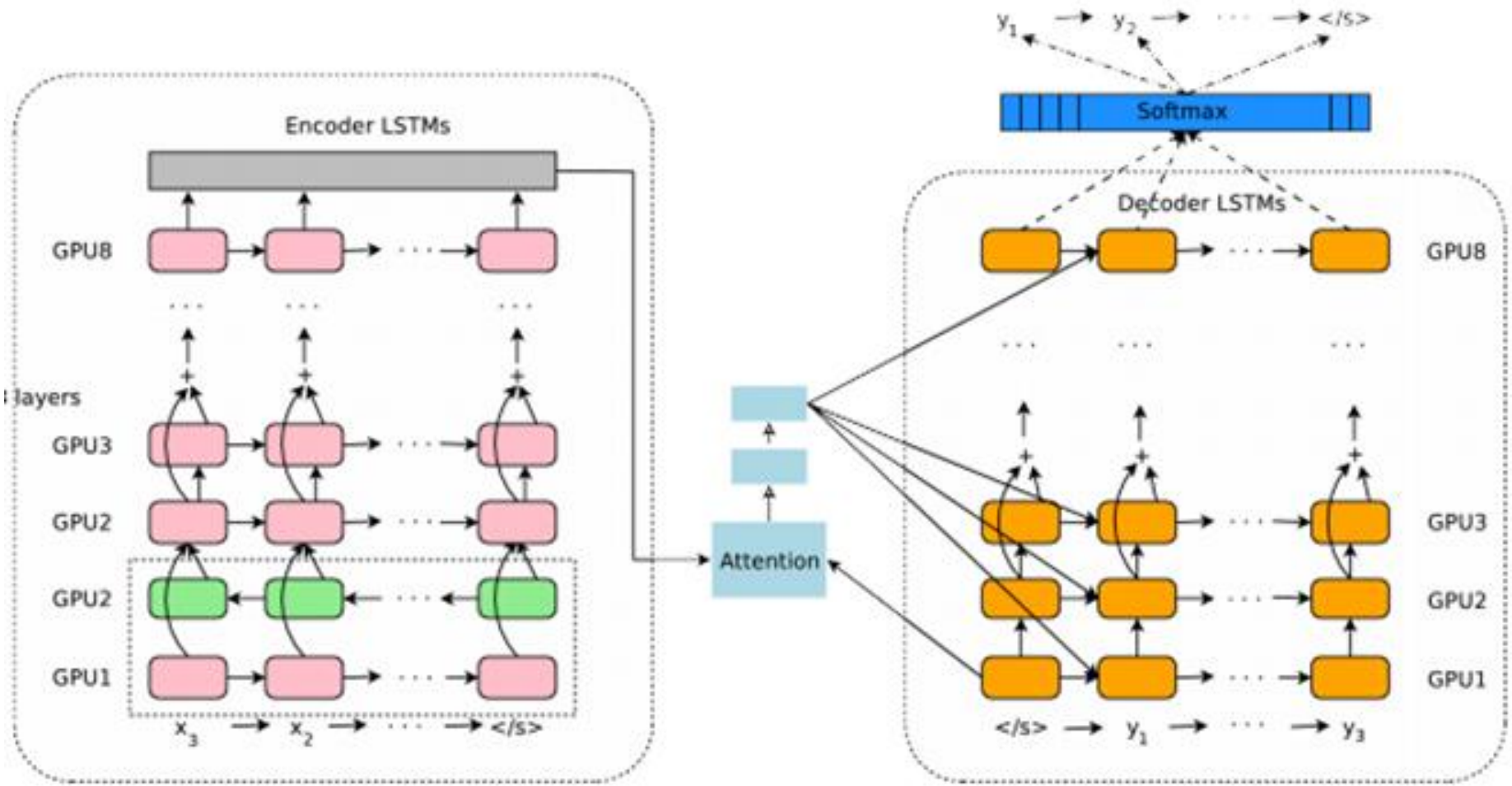
Google Neural Machine Translation

GNMT Efficient Translation

Google Neural Machine Translation System (GNMT) adopts a deep architecture to process complex language structures, effectively handling tasks such as Chinese English translation, and achieving efficient and accurate automated translation services.

Efficient processing mechanism

GNMT significantly improves translation speed and quality through parallel processing and low precision algorithm optimization, while introducing attention mechanism to ensure translation accuracy, bringing users a convenient international communication experience.


Encoder LSTMs
GPU8
Layers
GPU3
GPU2
GPU2
GPU1
x3
x2
</s>
Attention
y1
y2
</s>
Softmax
Decoder LSTMs
GPU8
GPU3
GPU2
GPU1
</s>
y1
y3

Design of min GRU/min LSTM

Design of minGRU/LSTM

In response to the limitations of traditional GRU/LSTM, minGRU/minLSTM is designed to improve training efficiency by simplifying the structure and parallel processing, while solving the problem of gradient vanishing and optimizing long-term dependency processing.

Gradient vanishing solution

minGRU/minLSTM effectively alleviates the gradient vanishing problem and improves model performance through a unique gating mechanism and parallel scanning algorithm, providing strong support for sequence data processing and promoting progress in NLP tasks.

Performance comparison and optimization

01

MinGRU/LSTM performance

minGRU and minLSTM demonstrate excellent performance in various NLP tasks, with simplified parameters and efficient training. Their performance is comparable to or even better than traditional GRU/LSTM models.

02

Task performance comparison

minGRU/minLSTM performs excellently in tasks such as selective replication, reinforcement learning, and language modeling, and is comparable in performance to advanced baseline models such as Mamba S6 and Decision Transformer.

GRU

$$
\begin{aligned}
\boldsymbol{h}_t &= (\mathbf{1} - \boldsymbol{z}_t) \odot \boldsymbol{h}_{t-1} + \boldsymbol{z}_t \odot \tilde{\boldsymbol{h}}_t \\
\boldsymbol{z}_t &= \sigma(\text{Linear}_{d_h}([\boldsymbol{x}_t, \boldsymbol{h}_{t-1}])) \\
\boldsymbol{r}_t &= \sigma(\text{Linear}_{d_h}([\boldsymbol{x}_t, \boldsymbol{h}_{t-1}])) \\
\tilde{\boldsymbol{h}}_t &= \tanh(\text{Linear}_{d_h}([\boldsymbol{x}_t, \boldsymbol{r}_t \odot \boldsymbol{h}_{t-1}]))
\end{aligned}
$$

$\Rightarrow$

minGRU

$$
\begin{aligned}
\boldsymbol{h}_t &= (\mathbf{1} - \boldsymbol{z}_t) \odot \boldsymbol{h}_{t-1} + \boldsymbol{z}_t \odot \tilde{\boldsymbol{h}}_t \\
\boldsymbol{z}_t &= \sigma(\text{Linear}_{d_h}(\boldsymbol{x}_t)) \\
\tilde{\boldsymbol{h}}_t &= \text{Linear}_{d_h}(\boldsymbol{x}_t)
\end{aligned}
$$

LSTM

$$
\begin{aligned}
\boldsymbol{h}_t &= \boldsymbol{o}_t \odot \tanh(\boldsymbol{c}_t) \\
\boldsymbol{o}_t &= \sigma(\text{Linear}_{d_h}([\boldsymbol{x}_t, \boldsymbol{h}_{t-1}])) \\
\boldsymbol{c}_t &= \boldsymbol{f}_t \odot \boldsymbol{c}_{t-1} + \boldsymbol{i}_t \odot \tilde{\boldsymbol{c}}_t \\
\boldsymbol{f}_t &= \sigma(\text{Linear}_{d_h}([\boldsymbol{x}_t, \boldsymbol{h}_{t-1}])) \\
\boldsymbol{i}_t &= \sigma(\text{Linear}_{d_h}([\boldsymbol{x}_t, \boldsymbol{h}_{t-1}])) \\
\tilde{\boldsymbol{c}}_t &= \tanh(\text{Linear}_{d_h}([\boldsymbol{x}_t, \boldsymbol{h}_{t-1}]))
\end{aligned}
$$

$\Rightarrow$

minLSTM

$$
\begin{aligned}
\boldsymbol{h}_t &= \boldsymbol{f}'_t \odot \boldsymbol{h}_{t-1} + \boldsymbol{i}'_t \odot \tilde{\boldsymbol{h}}_t \\
\boldsymbol{f}_t &= \sigma(\text{Linear}_{d_h}(\boldsymbol{x}_t)) \\
\boldsymbol{i}_t &= \sigma(\text{Linear}_{d_h}(\boldsymbol{x}_t)) \\
\tilde{\boldsymbol{h}}_t &= \text{Linear}_{d_h}(\boldsymbol{x}_t) \\
\boldsymbol{f}'_t, \boldsymbol{i}'_t &\leftarrow \frac{\boldsymbol{f}_t}{\boldsymbol{f}_t + \boldsymbol{i}_t}, \frac{\boldsymbol{i}_t}{\boldsymbol{f}_t + \boldsymbol{i}_t}
\end{aligned}
$$

Mamba Neural Network

01

Basic Principles

Structured State Space Model



State evolution matrix

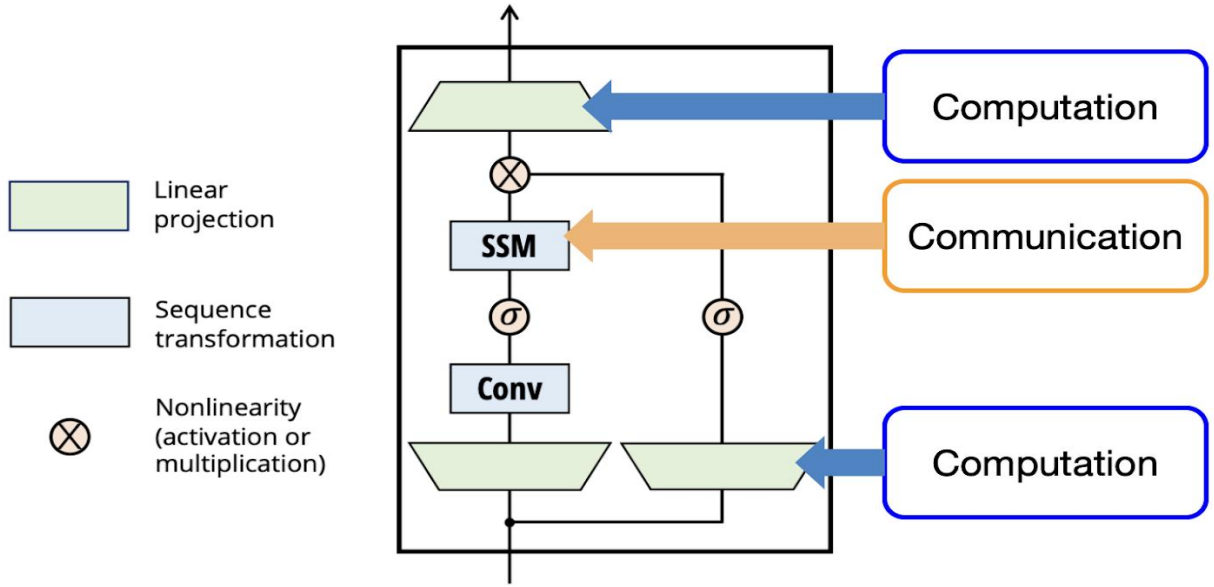
At the heart of the structured state space model lies the state evolution matrix, which governs the transition of states from one time step to the next.

Input dependency

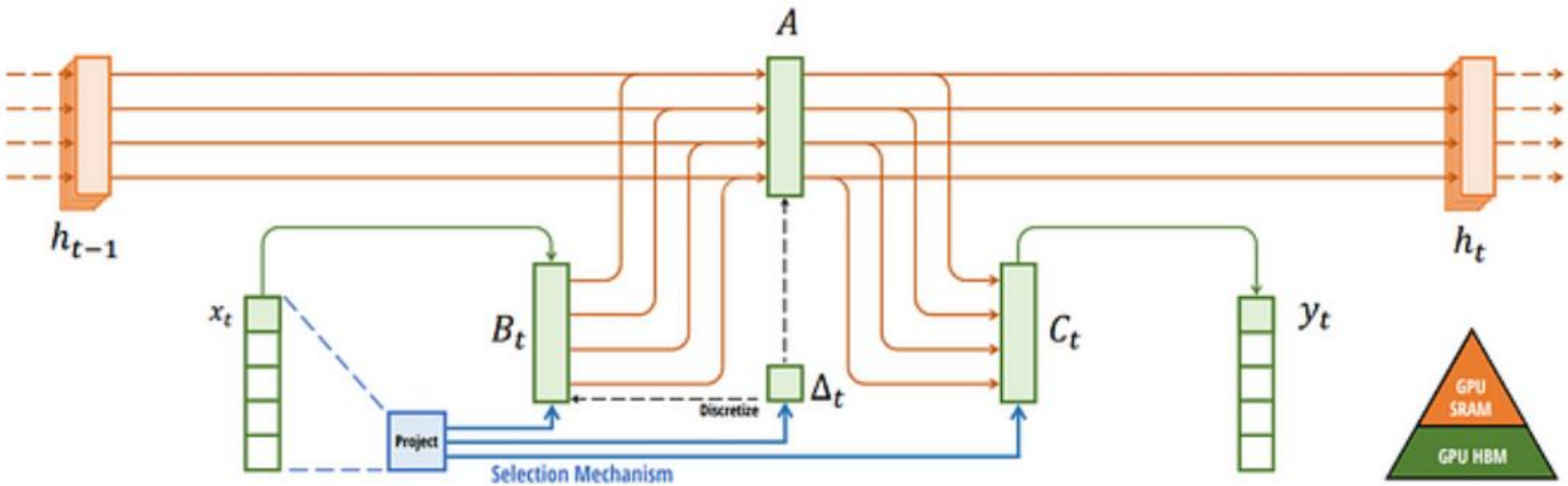
Inputs are crucial in driving state transitions. Input-dependent matrices capture the effect of inputs on state evolution, allowing the model to respond dynamically to changing input conditions.

Observation model

The observation model determines how states are observed or measured. It defines the relationship between the underlying states and the observable data, incorporating noise and other factors.


Linear projection
Sequence transformation
Nonlinearity (activation or multiplication)
SSM
Conv
Computation
Communication
Computation

Mamba


A
h_{t-1}
h_t
x_t
B_t
C_t
y_t
Δ_t
Discretize
Project
Selection Mechanism
GPU SRAM
GPU HBM

Linear time series modeling

Linear modeling simplicity

Linear time series models offer simplicity and interpretability in capturing linear relationships between inputs and outputs over time, assuming linearity and additivity.

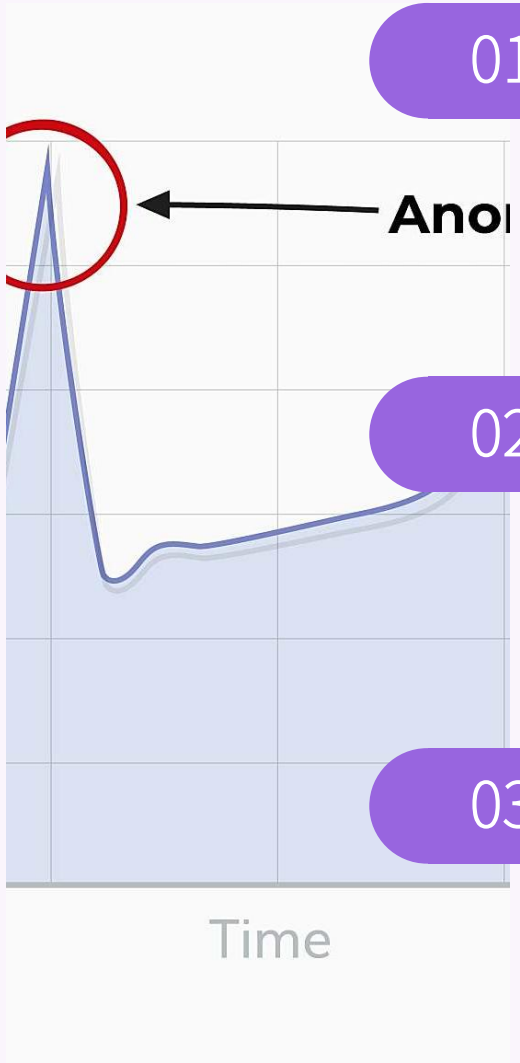
Stationarity assumption

Traditional linear models often assume stationarity, where the statistical properties of the data remain constant over time. This assumption simplifies model estimation and prediction.

Limitations of linearity

While linear models are easy to interpret and implement, they may have difficulty capturing complex, nonlinear relationships that are common in real-world data.

Dynamic parameter adaptation mechanism



01

Input-dependent parameters

Mamba generates parameters dynamically based on inputs, in contrast to traditional SSMs with fixed parameters. This enables the model to adapt more flexibly to changing sequence contexts.

02

Selective mechanism

Mamba retains only critical information while filtering out irrelevant details, improving computational efficiency. This selective mechanism allows the model to focus on what is important, reducing redundancy.

03

Linear complexity

Mamba achieves linear complexity through the use of zero-order hold discretization and parallel scanning algorithms. This makes it efficient for handling long sequences, ensuring scalable performance.

02

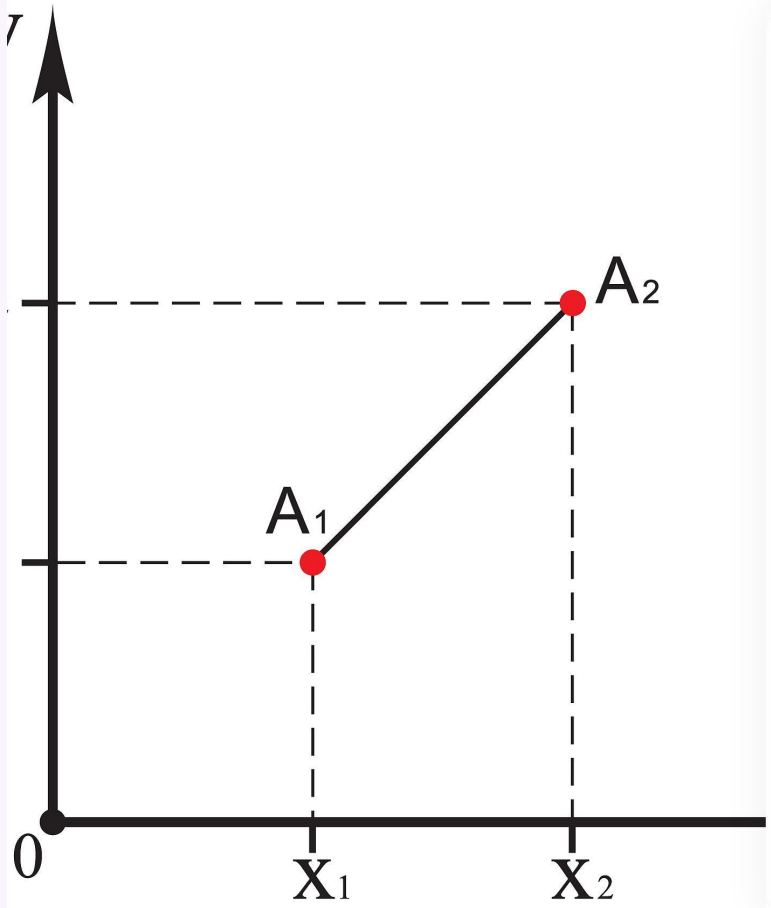
Core formula

Continuous-time SSM:

$$h_t = Ah_{t-1} + B(u_t)u_t,\ y_t = C(u_t)h_t + Du_t$$

Here, A is a state transition matrix, and $B(u_t), C(u_t)$ are input-dependent matrices.

Continuous time equation of state



State evolution

In the continuous-time state space model, the state vector s(t) evolves according to a differential equation driven by the state transition matrix A and input-dependent matrices B.

Input dynamics

The input-dependent parameters in Mamba generate time-varying state transition and input matrices, allowing the model to dynamically adapt to the sequence context.

Parameter generation (input-dependent):

$$\Delta_t = \tau_\Delta(W_\Delta x_t), B_t = W_B x_t, C_t = W_C x_t$$

where τ_Δ is an activation function (e.g., softplus), and W_Δ, W_B, W_C are learnable weights.

Input relevant parameters to generate

Parameter generation

In Mamba, input-dependent parameters are generated using a combination of learnable weights and activation functions, such as softplus, to introduce non-linearity.

Adaptive behavior

The generated parameters, which are different from traditional SSMs with fixed parameters, enable Mamba to adapt more dynamically to sequence context.

Discretization (ZOH method):

$$\bar{A} = e^{\Delta_t A}, \bar{B} = (\Delta_t A)^{-1}(e^{\Delta_t A} - I)\Delta_t B_t$$

This converts continuous dynamics to discrete steps for numerical computation.

Zero order preservation discretization

01

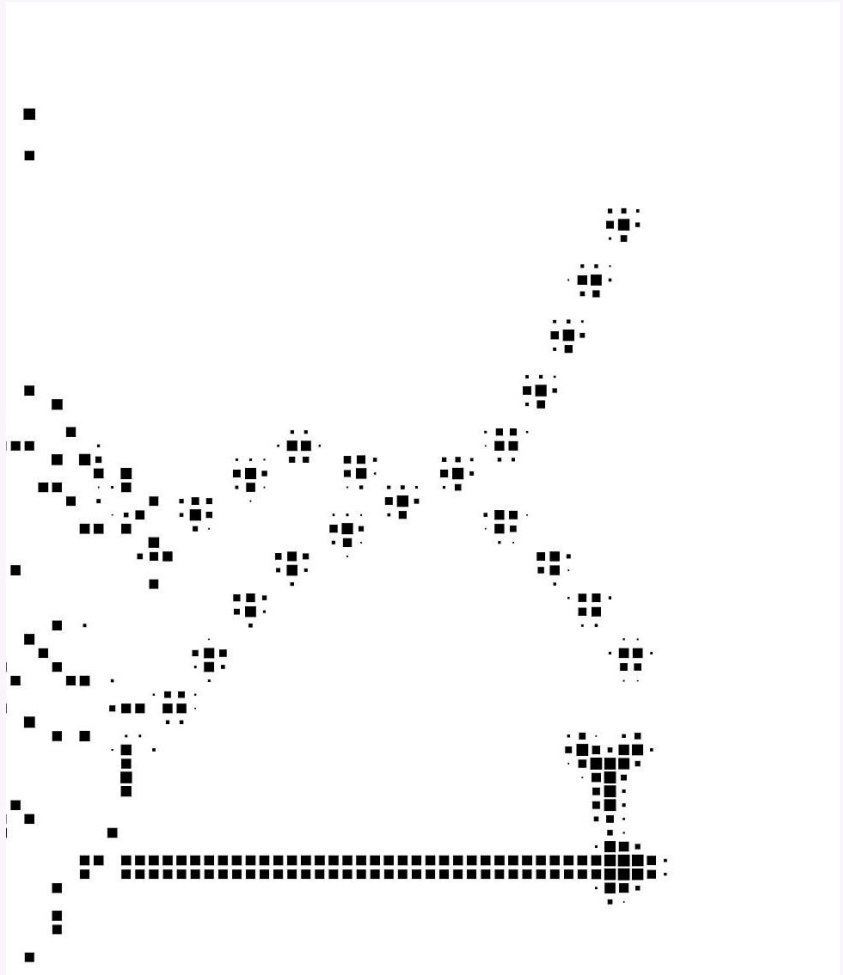
ZOH discretization

The zero-order hold (ZOH) method is used to convert the continuous-time dynamics of the state space model into a discrete-time update equation for numerical computation.

02

Efficient computation

By using the ZOH method and parallel scanning algorithms, Mamba achieves linear complexity for both training and inference, making it efficient for long sequences.



Discrete state update:

$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t u_t, y_t = C_t h_t + D u_t$$

The parallel scan algorithm enables efficient computation of all states simultaneously.

Parallel scanning status update

Parallel scan update

The parallel scan algorithm in Mamba enables the efficient computation of all states simultaneously by leveraging parallel processing capabilities.

Selective mechanism

The selective mechanism in Mamba retains critical information while filtering out irrelevant details, improving the efficiency of the model.

03

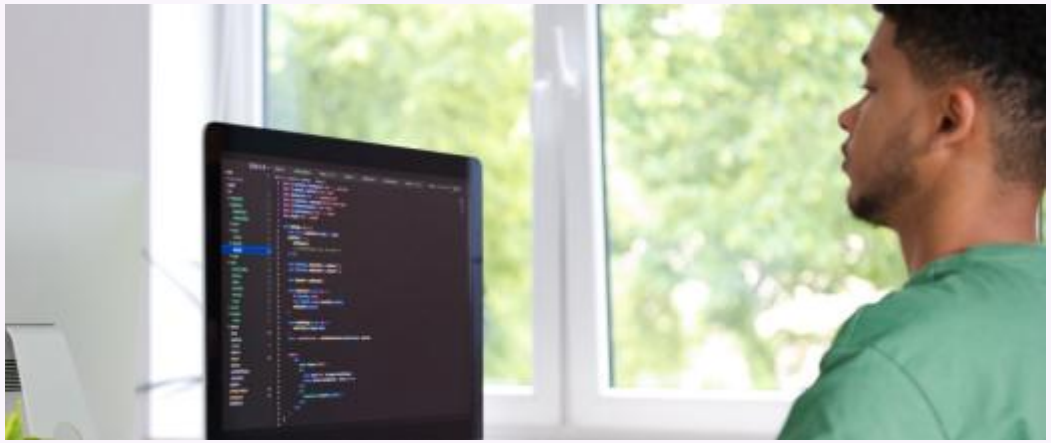
Calculation Example

Parameter Setting for Scalar Cases



Simplified scalar case

For the sake of simplicity, consider a scalar case with an initial state of x_0 and an input sequence u. This serves as a foundational example.



Fixed parameters

To keep the example manageable, let's fix some parameters for this scalar case. Set A = 0.5 and B = 1.0. These values will be used in the calculations.

Discretization Step Demonstration

01

Zero-order hold discretization

To convert the continuous-time dynamics into discrete steps, zero-order hold (ZOH) discretization is employed. This method is straightforward.

02

Parameter computation

After defining the fixed parameters A and B, we compute the discretized parameters by applying the ZOH discretization method. This step is crucial.

Detailed explanation of state recursion process

State update recursion

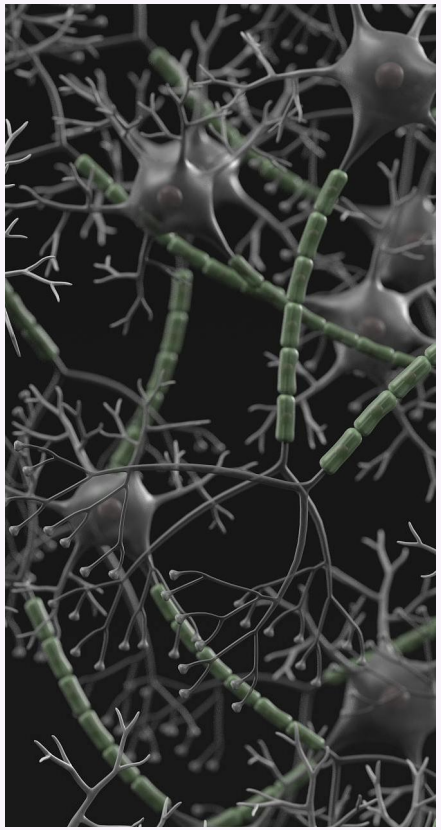
Starting from the initial state x_0 , we sequentially update the states of the system based on the input sequence u and the discretized parameters.

Parallel scan algorithm

Mamba's parallel scan algorithm enables us to efficiently compute all states simultaneously, leveraging linear complexity for long sequences.

Final output result analysis







Final states

After sequentially updating the states based on the input sequence and discretized parameters, we obtain the final states of the system.

Analysis of results

By examining the final states, we can gain insights into the behavior of the system over time and assess the effectiveness of the model.

Assume a simplified scalar case:

$A = -0.5$, initial state $h_0 = 0$, input sequence $u = [1, 2, 3]$.

Fixed parameters: $\Delta_t = 0.1$, $B_t = 1$ (for simplicity).

Step 1: Compute discretized parameters:

$$\bar{A} = e^{\Delta_t A} = e^{-0.05} \approx 0.9512, \bar{B} = \frac{e^{\Delta_t A} - 1}{\Delta_t A} \cdot \Delta_t B_t \approx 0.0975$$

Step 2: Update states sequentially:

For $u_1 = 1$: $h_1 = \bar{A} \cdot 0 + \bar{B} \cdot 1 \approx 0.0975$

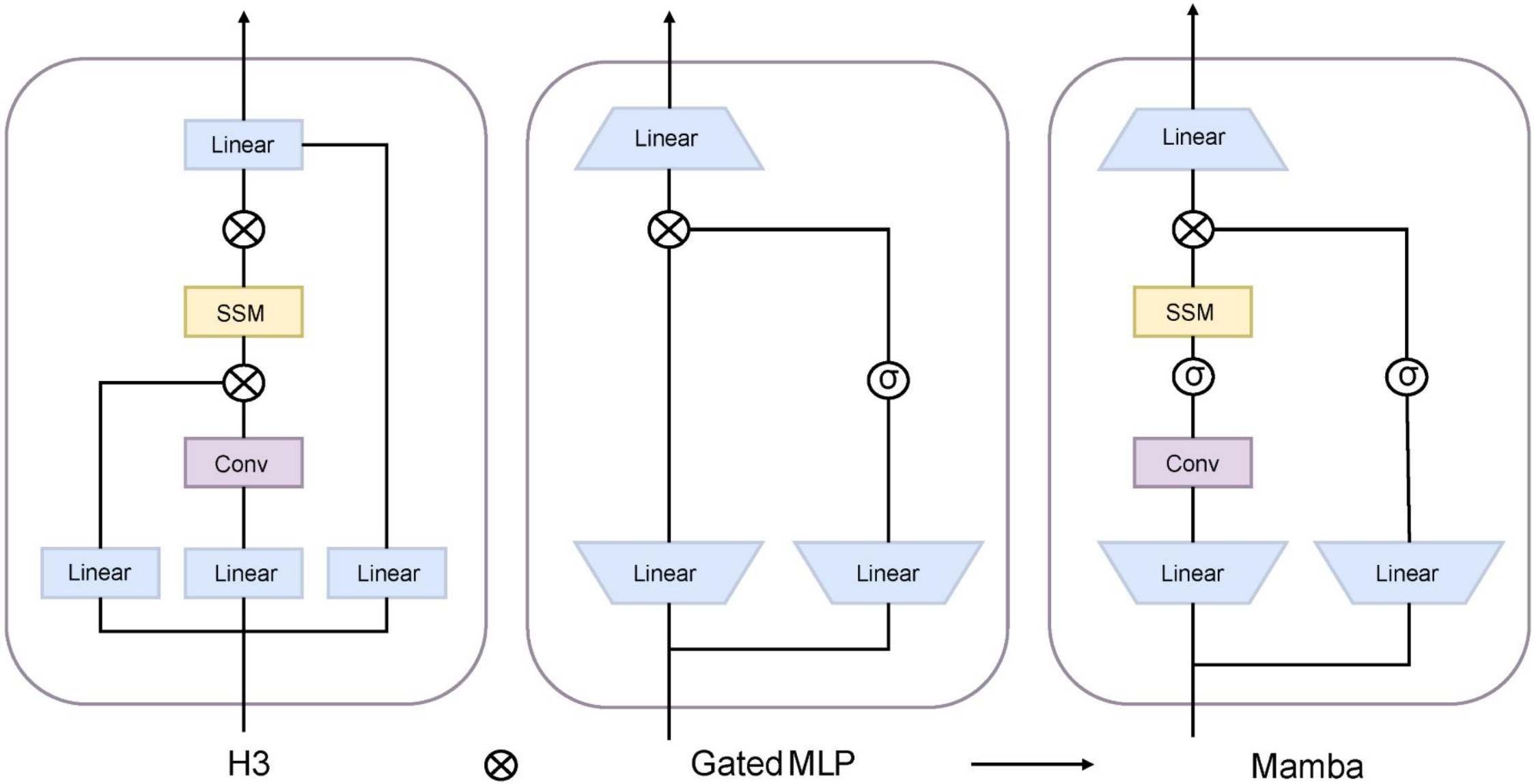
For $u_2 = 2$: $h_2 = \bar{A} \cdot 0.0975 + \bar{B} \cdot 2 \approx 0.2879$

For $u_3 = 3$: $h_3 = \bar{A} \cdot 0.2879 + \bar{B} \cdot 3 \approx 0.5665$

Final states: $[0.0975, 0.2879, 0.5665]$.

04

Technical advantage


Linear
SSM
Conv
Linear
Linear
Linear
H3
⊗
Linear
σ
Linear
Linear
Gated MLP
Linear
SSM
σ
σ
Conv
Linear
Linear
Mamba

Efficiency of Long Sequence Modeling

- Advantages of Linear Complexity: Mamba exhibits significant advantages in linear complexity when processing long sequences, effectively reducing computational resources and time consumption compared to the quadratic complexity of traditional Transformers.
- Bioinformatics applications: In the field of bioinformatics, Mamba's ability to process long sequences is particularly critical. For example, in the process of DNA sequence analysis, Mamba can efficiently process large-scale data and accurately predict gene structure.
- Meteorological prediction: When used for meteorological prediction, Mamba's efficient long sequence processing capability helps the model accurately predict future climate change trends, providing strong support for disaster prevention and mitigation, and ensuring social security and stability.
- Financial analysis: In the field of financial analysis, Mamba can efficiently process massive market data, accurately capture price trends, provide strong support for investment decisions, and help investors achieve maximum returns.

Selective attention mechanism

- Selective attention mechanism: The core of Mamba lies in the selective attention mechanism, which can automatically filter out irrelevant information, significantly improving the processing efficiency of the model and ensuring more accurate decision-making.
- Information bottleneck theory: According to the information bottleneck theory, Mamba's selective attention mechanism can automatically learn and retain key information while suppressing interference from irrelevant information, thereby improving overall processing efficiency.
- Biological insights: From a biological perspective, Mamba's selective attention mechanism is inspired by the selection mechanism of organisms when processing massive amounts of information, which can efficiently identify and respond to key information.
- Application effect improvement: Selective attention mechanism plays a key role in Mamba, significantly improving the accuracy and efficiency of the model in processing complex tasks by automatically filtering irrelevant information.

Principle of Linear Complexity

- The principle of linear complexity: Mamba effectively reduces the computational and time consumption of the model when processing long sequence data by adopting a design with linear complexity, thereby significantly improving efficiency.
- Numerical stability: Compared to traditional models with quadratic complexity, Mamba's linear complexity design significantly enhances numerical stability and reduces computational resource requirements while maintaining high-precision predictions.
- Application advantages: With its linear complexity design, Mamba demonstrates great potential in application scenarios that require processing large-scale datasets, and can efficiently handle various complex data processing tasks.
- Future outlook: With the continuous growth of data volume, Mamba's linear complexity design will bring it a wider range of application prospects, ensuring that the model can continue to play an important role in the future big data environment.

Comparison of actual application scenarios

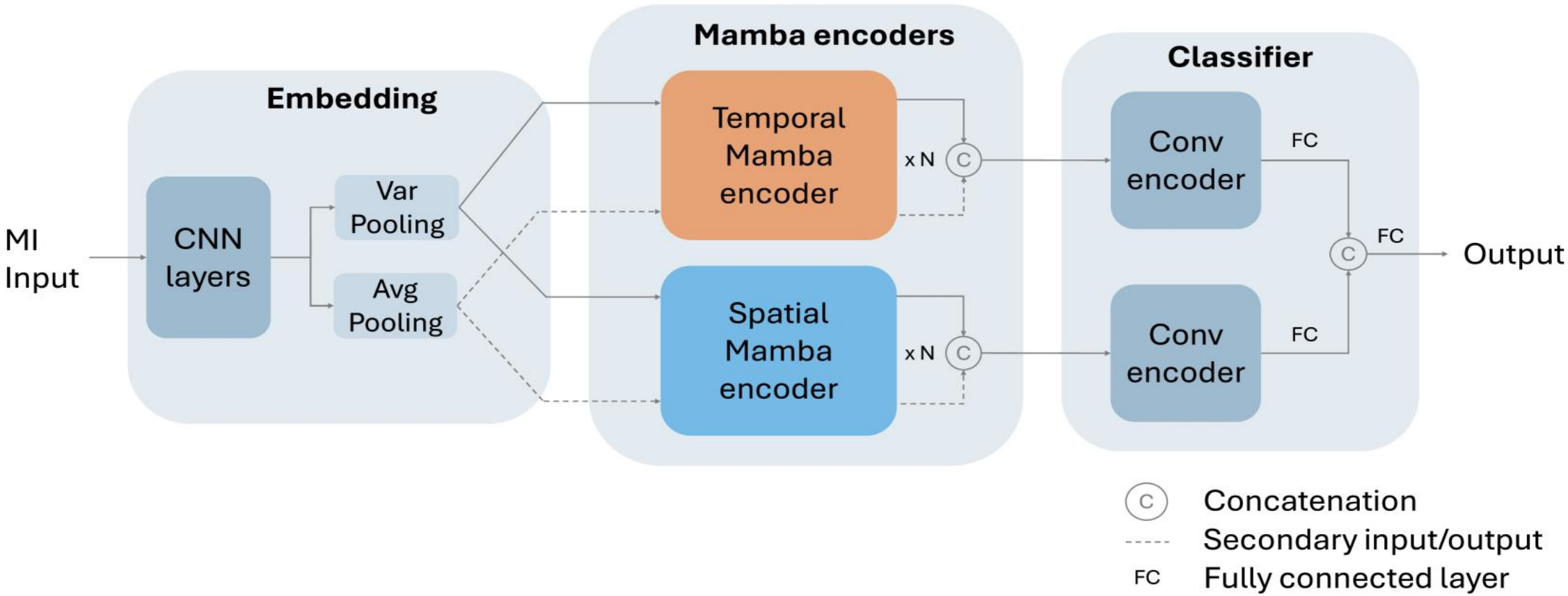
- Sequence to sequence learning: In sequence to sequence learning tasks, Mamba, with its excellent ability to handle long sequences, easily addresses the challenge of varying input and output sequence lengths, demonstrating better performance than traditional RNNs and CNNs.
- Point process prediction: Mamba performs well in point process prediction, efficiently processing and predicting events at various time points in the sequence. Its accuracy and efficiency are superior to other mainstream prediction models, providing strong support for point process analysis.
- Variable length sequence: For variable length sequence data, Mamba demonstrates strong adaptability by flexibly adjusting the model structure to match input sequences of different lengths, while maintaining high levels of prediction accuracy and efficiency.
- Multi domain applications: Mamba's broad application potential is not limited to a single domain, but is applicable to complex sequence processing tasks in multiple domains. Its unique architecture and efficient performance provide possibilities for cross domain applications.

05

Architectural innovation points

RNN and CNN feature fusion

- RNN long-term dependency capture: Mamba integrates RNN and CNN to accurately capture sequence long-term dependencies. By carefully designing loop and convolution structures, historical information is efficiently extracted, enhancing sequence modeling capabilities, and providing powerful tools for complex time-series data analysis.
- CNN parallel computing optimization: Introducing CNN parallel computing mechanism significantly improves processing efficiency. This strategy not only accelerates the model training and inference process, but also enhances the model's ability to utilize parallel resources, providing strong support for complex sequence tasks.
- Attention selection mechanism: Empowered by attention mechanism, accurately focusing on key information. Through selective attention, Mamba can efficiently filter irrelevant details, preserve core elements, optimize resource utilization, and improve sequence processing efficiency and accuracy.


Mamba encoders
Classifier
Embedding
Temporal Mamba encoder
x N
C
Conv encoder
FC
MI Input
CNN layers
Var Pooling
Avg Pooling
FC
C
Output
Spatial Mamba encoder
x N
C
Conv encoder
FC
C
Concatenation
Secondary input/output
FC
Fully connected layer

Dynamic parameter generation design

- Input driven parameter changes: Mamba innovatively introduces input dependent parameters to give the model dynamic adaptability. According to the changes in the input sequence, the parameters can be flexibly adjusted to ensure that the model closely fits the data characteristics and improve the flexibility and accuracy of sequence modeling.
- Efficiently capturing sequence changes: Accurately capturing sequence changes with dynamic parameters enhances modeling capabilities. By flexibly adjusting parameters, Mamba can more accurately capture key dynamic features in sequences, providing a solid foundation for tasks such as prediction and classification.
- Improving sequence modeling capability: Mamba's dynamic parameter design is like equipping the model with flexible adaptability, enabling it to easily cope with various complex sequence change challenges, thereby significantly improving the accuracy and efficiency of sequence modeling.

Parallel computing implementation scheme



Parallel computing efficiency improvement

Mamba adopts parallel scanning algorithm to achieve efficient utilization of computing resources. By simultaneously processing multiple state updates, the training and inference speed is significantly accelerated, providing a powerful processing solution for large-scale sequence data.



Linear complex management

Mamba's parallel computing solution is like injecting a powerful processing engine into the model, making it easy to cope with the challenges of large-scale sequential data, thereby significantly improving the speed and efficiency of the model and providing support for practical applications.



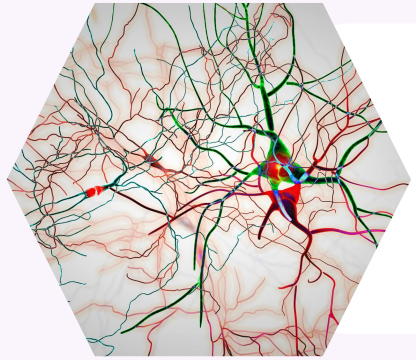
Scanning algorithm optimization

Through carefully optimized parallel scanning algorithms, Mamba effectively manages resource utilization while maintaining efficient computing, ensuring the stable operation of the model on large-scale data and providing reliable support for practical applications.

06

Performance verification

Genomics task performance



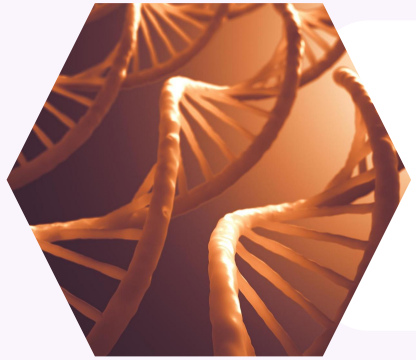
Mb-seq2seq

Mamba neural network significantly outperforms traditional Transformer-based models on genomics tasks, achieving state-of-the-art results on long-sequence prediction benchmarks.

Genomic prediction

Mamba's linear complexity and selective attention mechanism enable it to handle long genomic sequences with high accuracy, predicting gene expressions and regulatory elements with unprecedented precision.





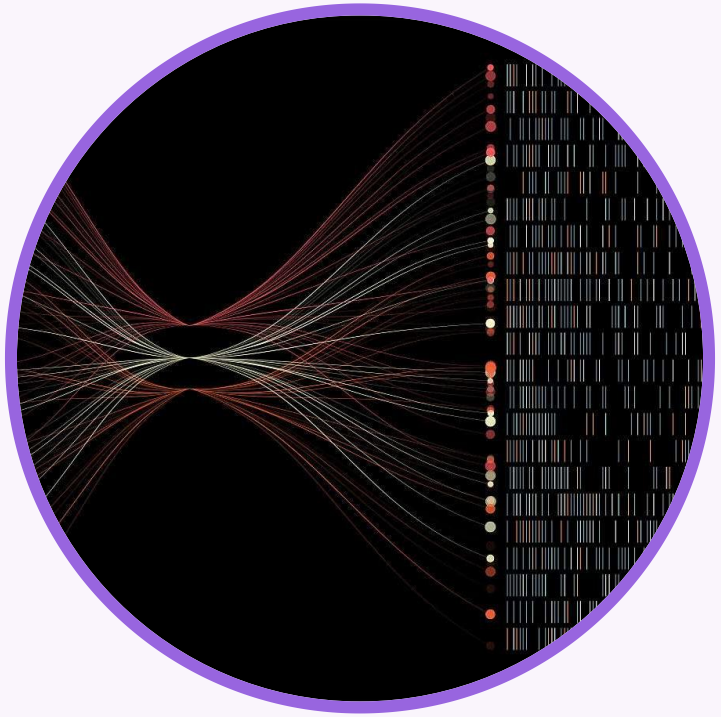
Computational efficiency

Thanks to its linear complexity, Mamba requires significantly less computational resources than Transformers for long-sequence tasks, making it a more efficient and scalable solution for genomics applications.

Spatiotemporal prediction accuracy

Mb-spatiotemporal

Mamba neural network demonstrates superior performance on spatiotemporal prediction tasks, accurately forecasting the dynamics of spatial processes over time with high precision.



Traffic congestion

In traffic congestion prediction, Mamba accurately predicts the likelihood of congestion at each time step, significantly outperforming traditional Transformer-based models.

Pollution levels

For air pollution level prediction, Mamba accurately predicts the concentration of pollutants at each time step, demonstrating its potential for supporting environmental management strategies.

Compared to traditional Transformers

Mb-vs-trad

Mamba consistently outperforms traditional Transformer models on long-sequence tasks, thanks to its linear complexity and selective attention mechanism, which are particularly effective in handling long contexts.

Acc-comp

While maintaining competitive accuracy with Transformers, Mamba achieves this with significantly less computational resources, thanks to its linear complexity and more efficient architecture.

Long-seq task

Mamba's advantage over Transformers is even more pronounced on tasks with extremely long sequences, such as genomic sequence analysis or spatiotemporal forecasting, where its linear complexity and selective attention mechanism allow it to handle the sequences with greater efficiency and accuracy.

07

implementation details

Activation function selection

1 Mamba activation function

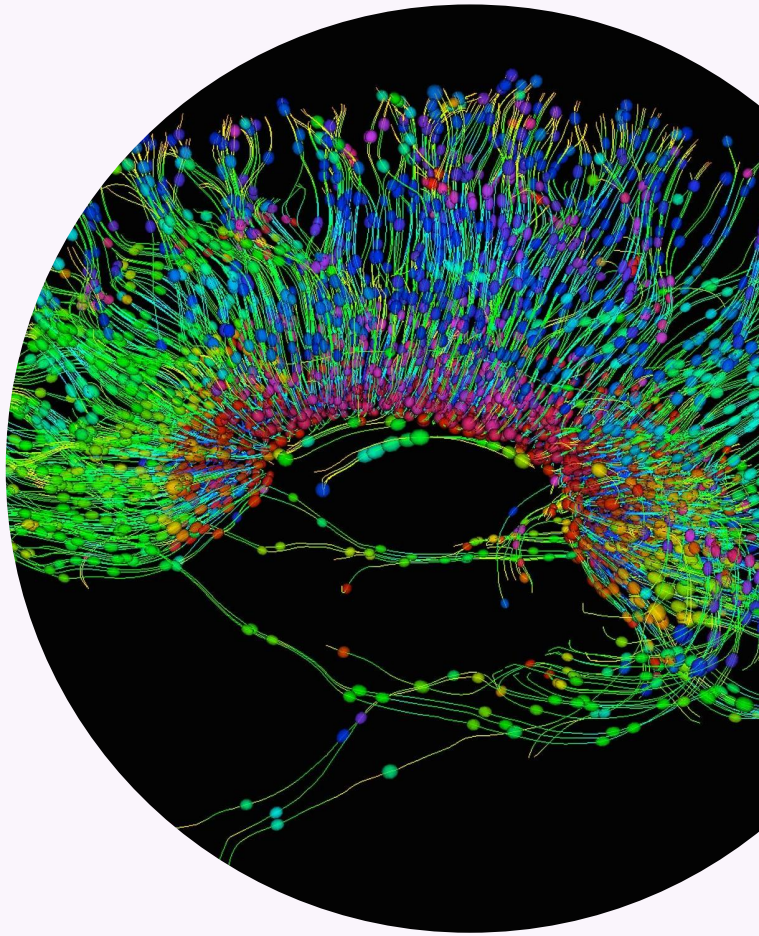
In the implementation of Mamba neural networks, we carefully selected the activation function to ensure that the network can learn and represent complex nonlinear relationships.

2 Parameterized matrix generation

Using activation functions (such as softplus) and learnable weights, the input is transformed into a parameterized matrix to dynamically adapt to the sequence context.

3 Efficient processing of long sequences

By using the zero order hold (ZOH) method to convert continuous dynamics into discrete steps, combined with parallel scanning algorithms, efficient processing of long sequences is achieved.



Weight initialization strategy

Mamba weight initialization

Weight initialization is crucial in Mamba neural network training as it affects gradient flow and model performance.

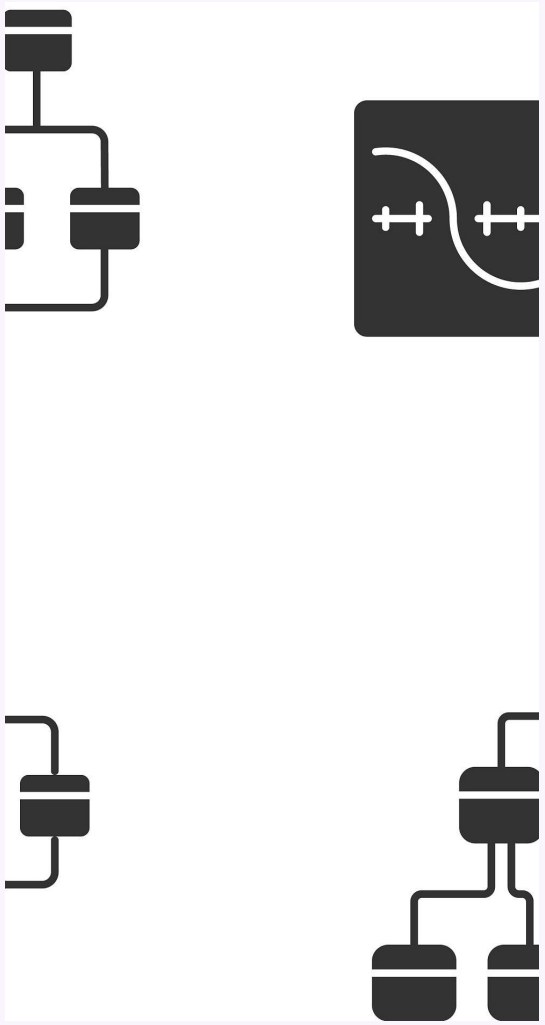
Weight initialization strategy

Adopting an initialization strategy based on random normal distribution to ensure that the initial weight values are within a certain range.

Adaptive adjustment

Based on the statistical characteristics of input data, adaptively adjust the weight initialization strategy to improve model training efficiency and performance.

Gradient computation optimization



Mamba gradient optimization

In the implementation of Mamba neural network, we optimized the calculation of gradients to improve the efficiency of model training.

Efficient gradient calculation

By using matrix differentiation and chain rule, efficient gradient calculation is achieved, reducing computational complexity and memory consumption.

Gradient clipping and updating

Introducing gradient clipping strategy to avoid gradient explosion or disappearance problems; Combining Adam and other optimization algorithms to achieve stable and efficient model training.

08

Extended Application

DNA sequence analysis

Efficient DNA analysis

Mamba excels in DNA sequence analysis, with its linear time complexity and selective attention mechanism capable of quickly and accurately identifying gene patterns.

Precision Medicine Promotion

Assisting in precision medicine, predicting disease risks through deep learning technology, implementing personalized treatment plans, and improving medical outcomes.

Advantages of sequence data

Mamba excels in processing long sequence data, such as gene sequences, and improves analysis efficiency and accuracy through parallel processing and selective attention.

Bioinformatics Applications

In bioinformatics, Mamba helps predict protein structures, analyze transcriptome data, and promote research and development in the field of life sciences.

Partial differential equation modeling

- Efficient modeling of PDE: Mamba performs well in partial differential equation (PDE) modeling, with its linear complexity and efficient parallel capability, making complex system simulations faster and more accurate.
- Precision Engineering Applications: In the field of precision engineering, Mamba assists in complex fluid simulation and structural optimization, improving design efficiency and product quality, and driving manufacturing towards higher precision.
- Speed up scientific computing: In scientific computing, Mamba accelerates numerical solution calculations, processes large-scale data efficiently and accurately, and provides powerful computing tools for researchers.
- Significant simulation optimization: The task of simulation optimization is heavy, and Mamba, with its efficient algorithms and parallel capabilities, significantly reduces time and assists in decision optimization and resource allocation.

Speech signal processing

Speech signal processing

Mamba optimizes speech signal feature extraction to enhance the accuracy of speech recognition, especially in noisy environments, improving the speech interaction experience.

Speech synthesis optimization

In speech synthesis, Mamba improves the naturalness and coherence of speech, optimizes text to speech through deep learning technology, and creates more realistic speech interactions.

Hearing aid upgrade

The hearing aid equipment adopts Mamba technology to accurately recognize speech features, providing a clear and smooth listening experience, bringing good news to people with hearing loss.

Speech data analysis

Mamba assists in speech big data analysis, improving data analysis efficiency and accuracy through deep mining, and providing intelligent support for speech recognition and applications.